

CAS Machine Learning – Time Series

Machine learning and deep learning
for time series

Dr. Ludovic Amruthalingam
ludovic.amruthalingam@hslu.ch

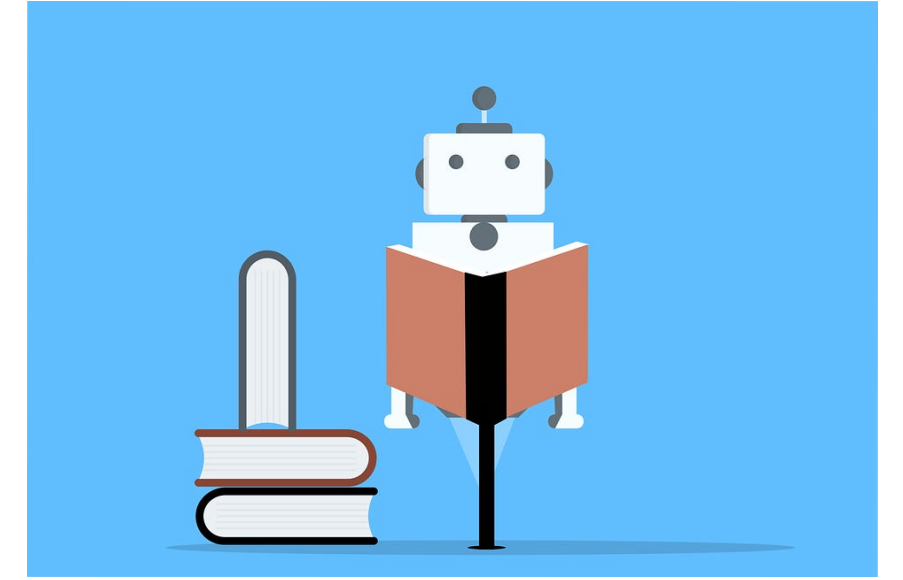
Informatik



Outline

- Machine learning
- Learning tasks
- Feature engineering
- Feature importance and selection
- Evaluation workflow
- Challenges
- Deep learning
- Convolutional neural networks
- Recurrent neural networks
- Practical tips

Machine learning



Machine learning studies **algorithms** that autonomously “learn” to perform **tasks** from **data** X .

$$t : X \rightarrow Y$$

Instead of manually defining specific steps and instructions, these algorithms are designed to automatically **extract patterns and statistical relationships** from data that are useful to complete the task.

$$m_{\theta} : X \rightarrow Y$$

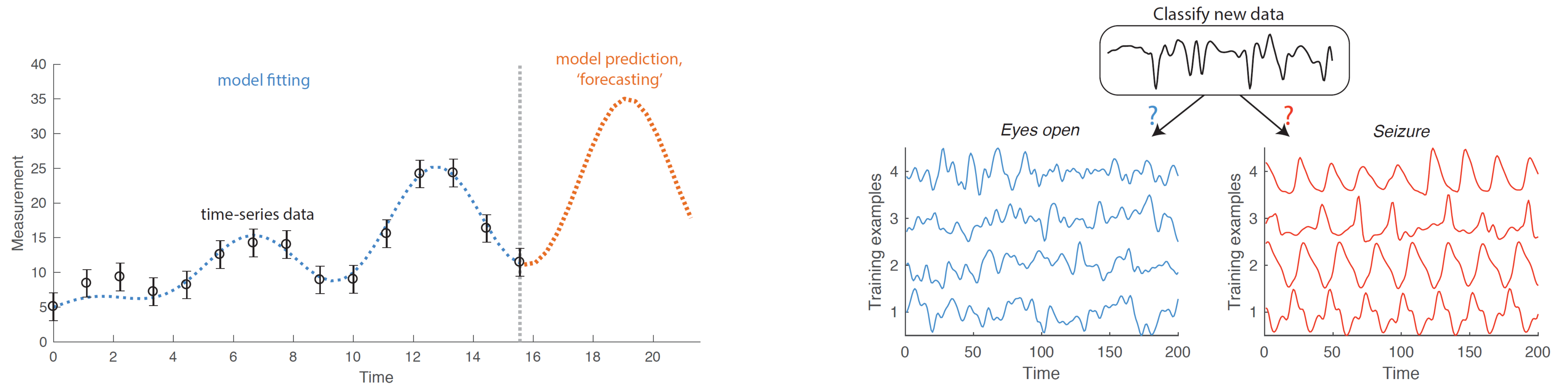
An algorithm is considered to “learn” if its **performance** improves as it processes data. The training procedure optimizes the parameters θ of the algorithm’s model m_{θ} with respect to a performance measure P :

$$\theta^* = \max_{\theta} \sum_{\forall x \in X} P(m_{\theta}(x), t(x))$$

Learning tasks – Supervised learning

Supervised learning: **both** the task **data** X **and target** Y are available.

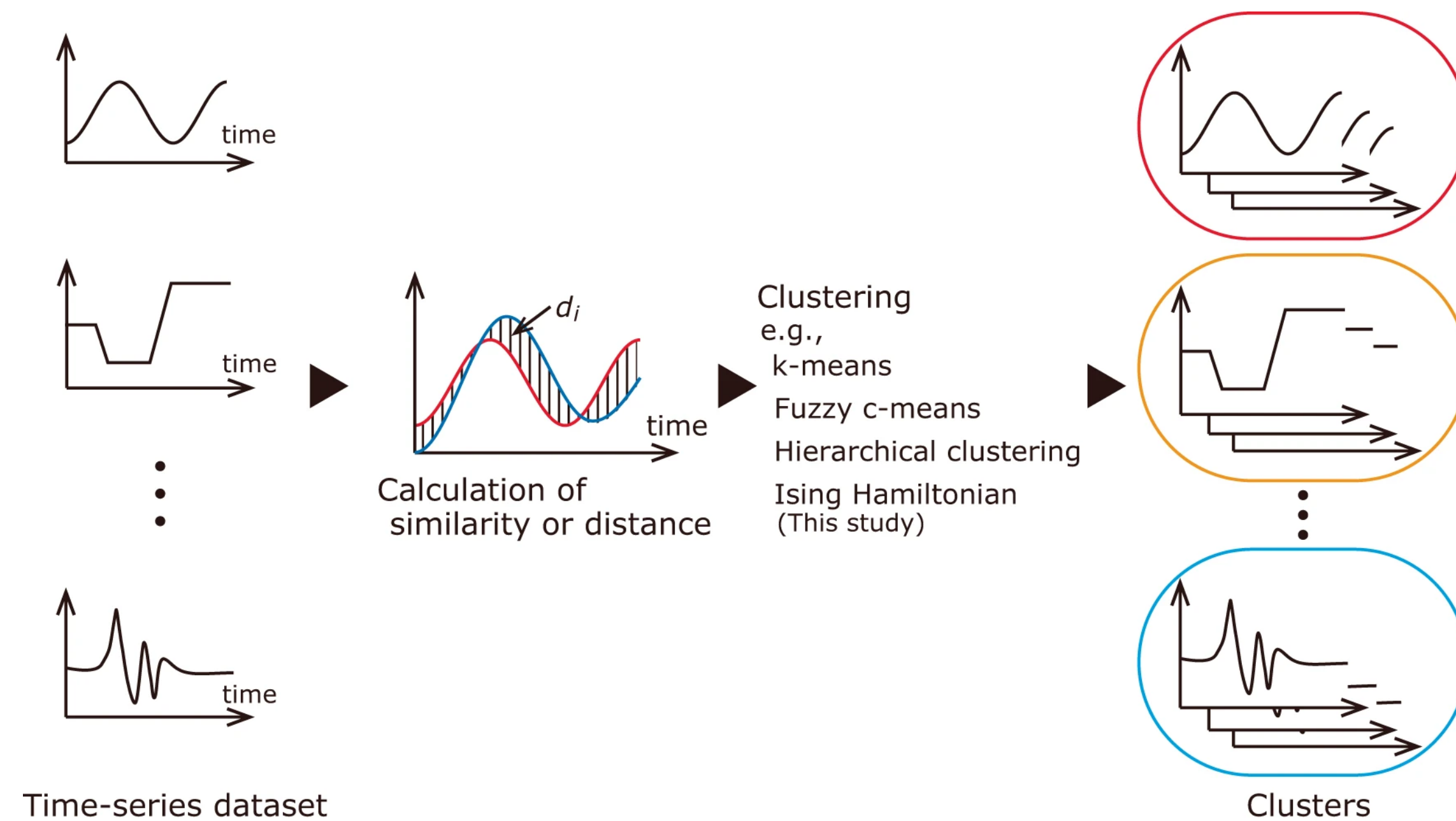
- Forecasting (Regression)
- Classification



Learning tasks – Unsupervised learning

Unsupervised learning: **only** the task **data** X is available.

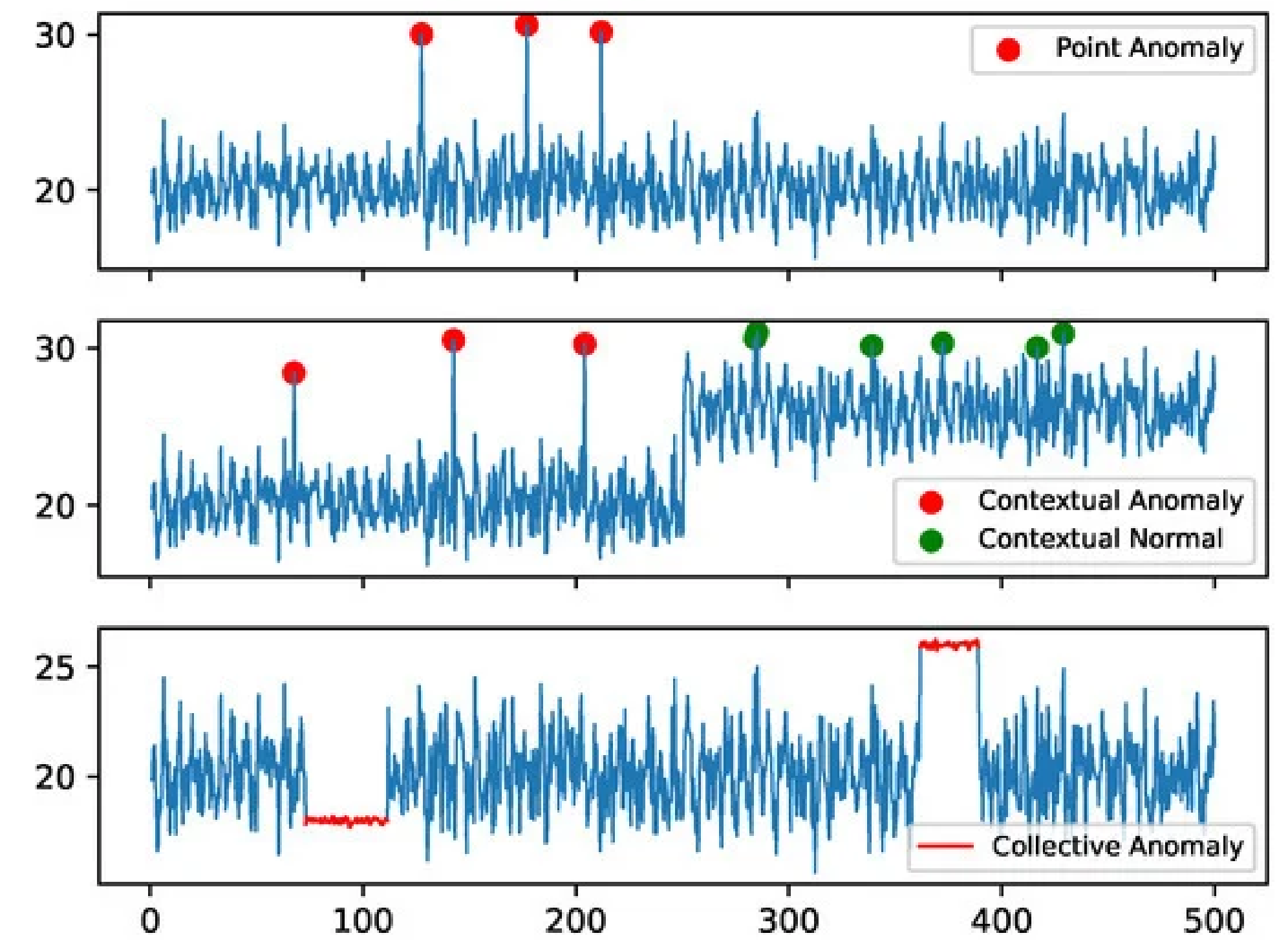
- Clustering
- Anomaly detection



Time-series dataset

Clusters

HSLU Inoue, Tomoki, et al. "Clustering method for time-series images using quantum-inspired digital annealer technology." (2024)



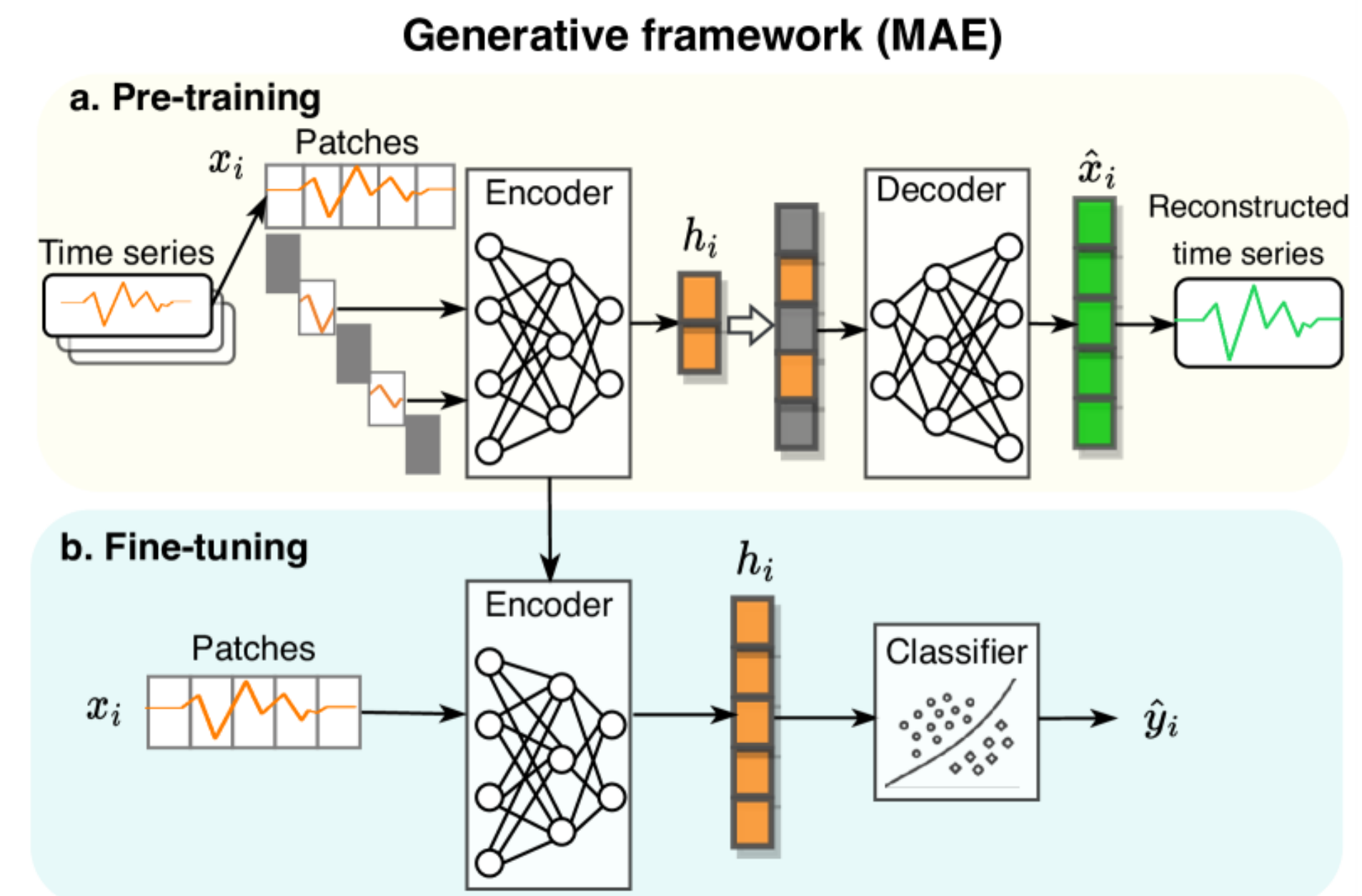
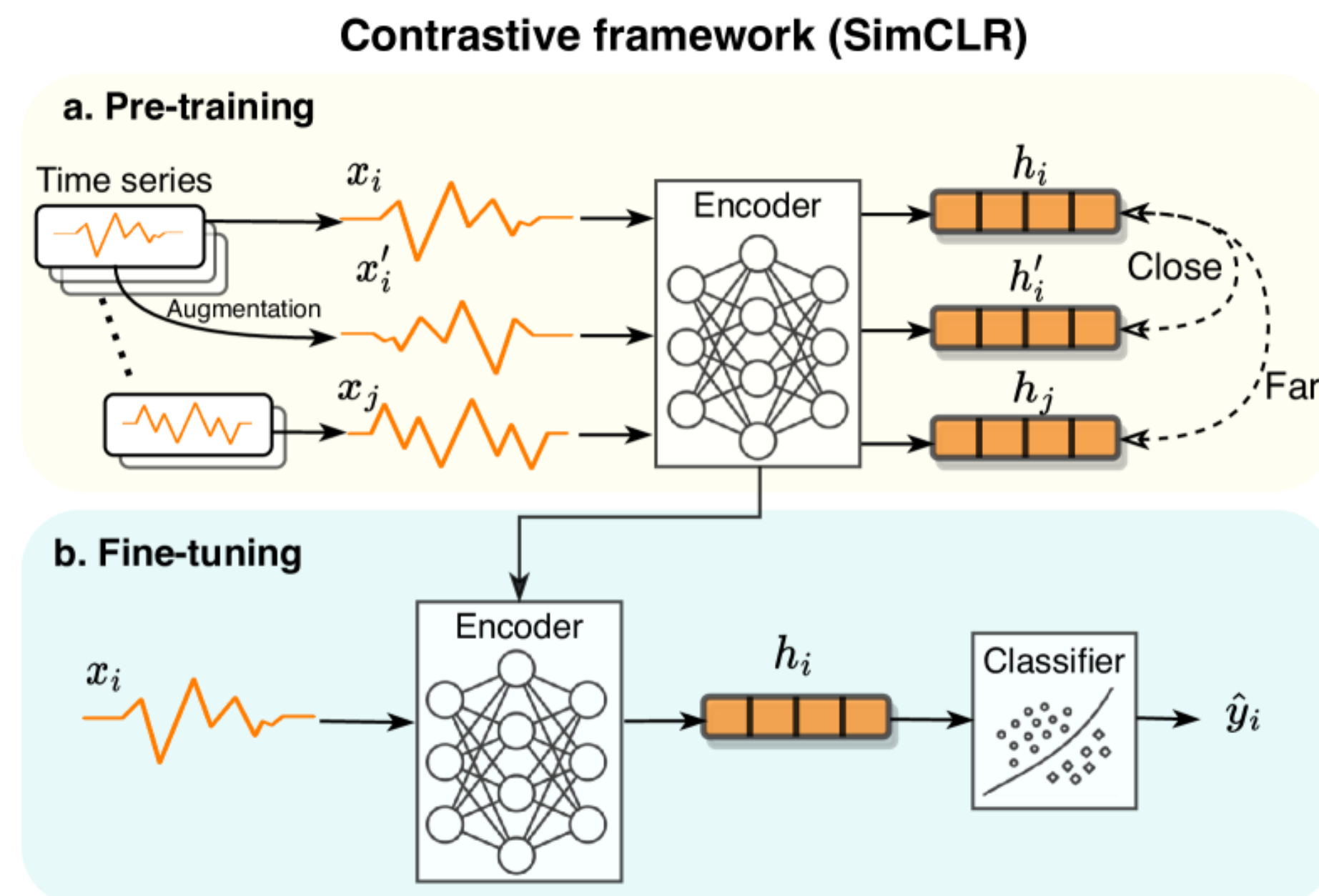
Belay, Mohammed Ayalew, et al. "Unsupervised anomaly detection for IoT-based multivariate time series: Existing solutions, performance analysis and future directions." (2023) Slide 5

Learning tasks – Variants

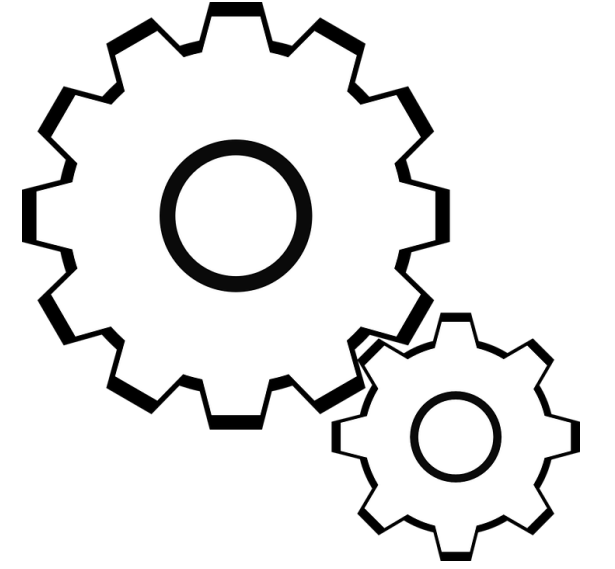
Semi-supervised learning

Self-supervised learning

Reinforcement learning: optimize agent's strategy to maximize a reward scheme.



Feature engineering for time series regression



ML models typically assume the samples are **independent and identically distributed (iid)**.

Time series data violates this assumption due to the **correlation between consecutive observations**.

- Each data point depends on previous time steps, creating temporal dependencies.

ML models do not directly incorporate **temporal dependencies** limiting their ability to fit time series patterns.

It is necessary to engineer features **capturing the temporal aspect** of the data

- Time: identify when events occurred e.g., weekday, month, year.
- Passage of time: elapsed time since a reference point.
- Periodicity of time: seasonality, cyclical encoding.

Lag features

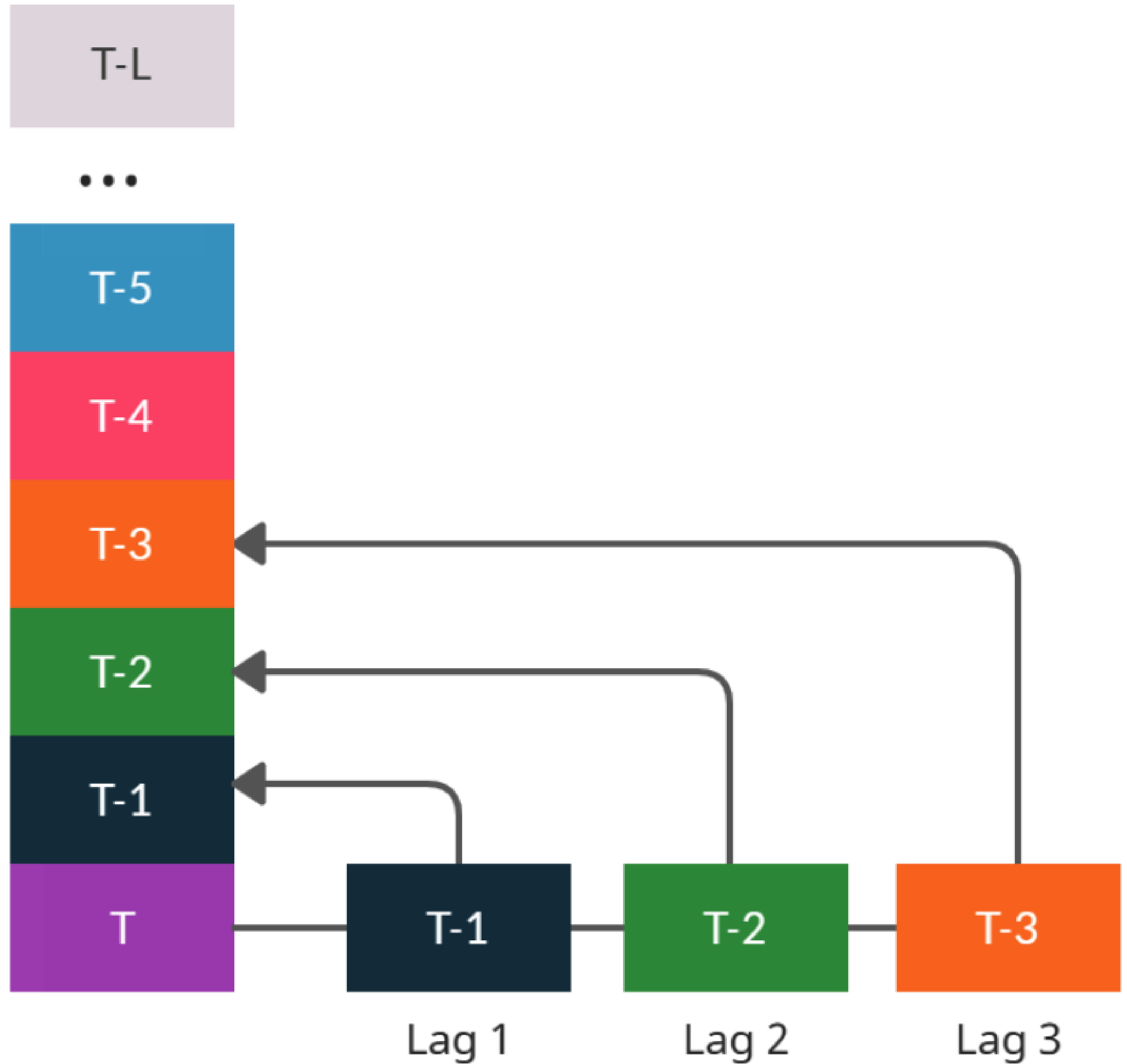
Use previous observations as features.

- Review ACF plot to identify the lags of interest.

Connect **present** observation with **past** observations.

pandas.DataFrame.shift

Current Timestamp



Rolling window aggregation

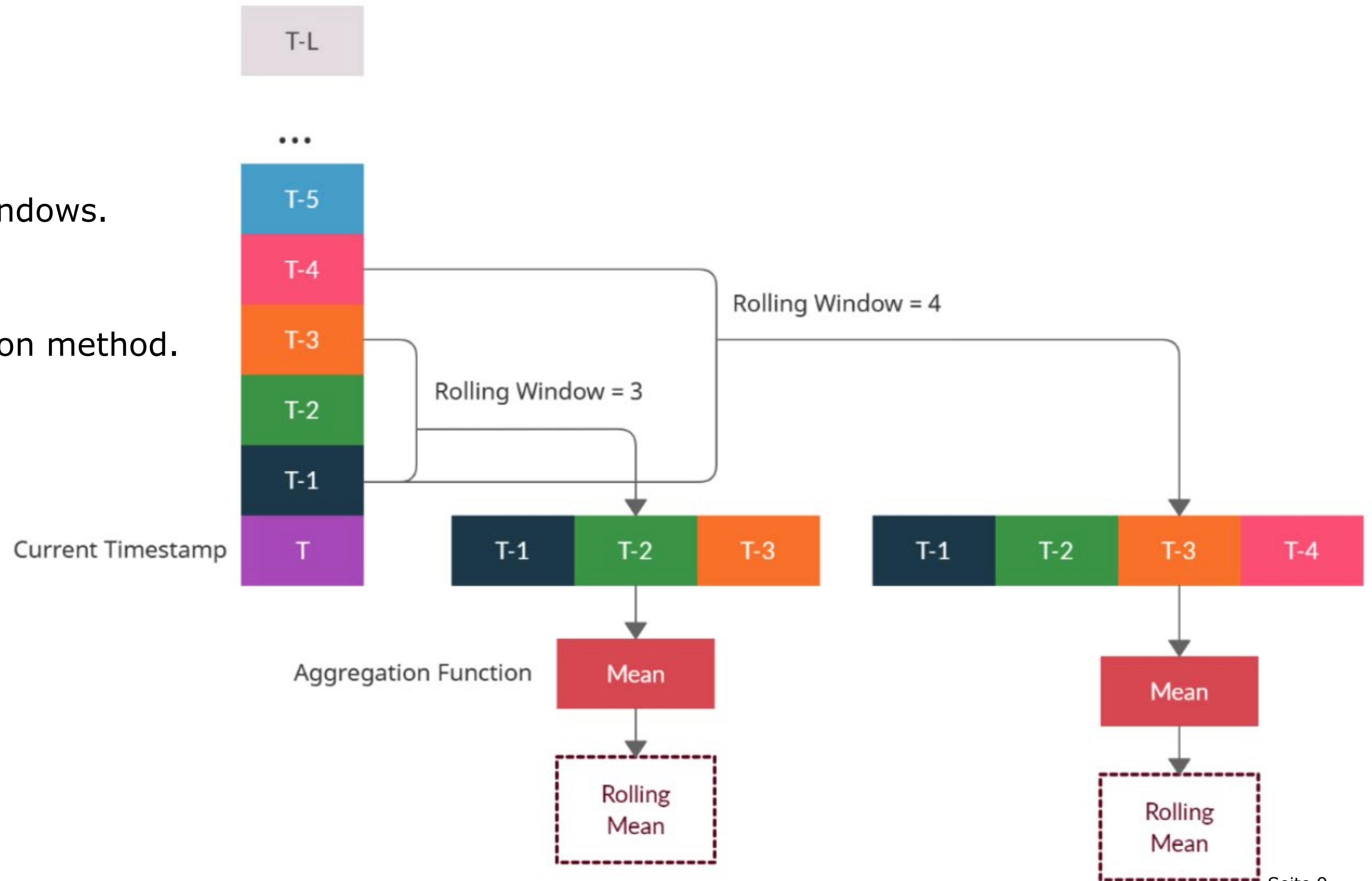
Rolling statistics with different windows.

Consider any **relevant** aggregation method.

- Mean, std, max, min, ...

Do **not** include x_t in the window!

→ data **leakage**



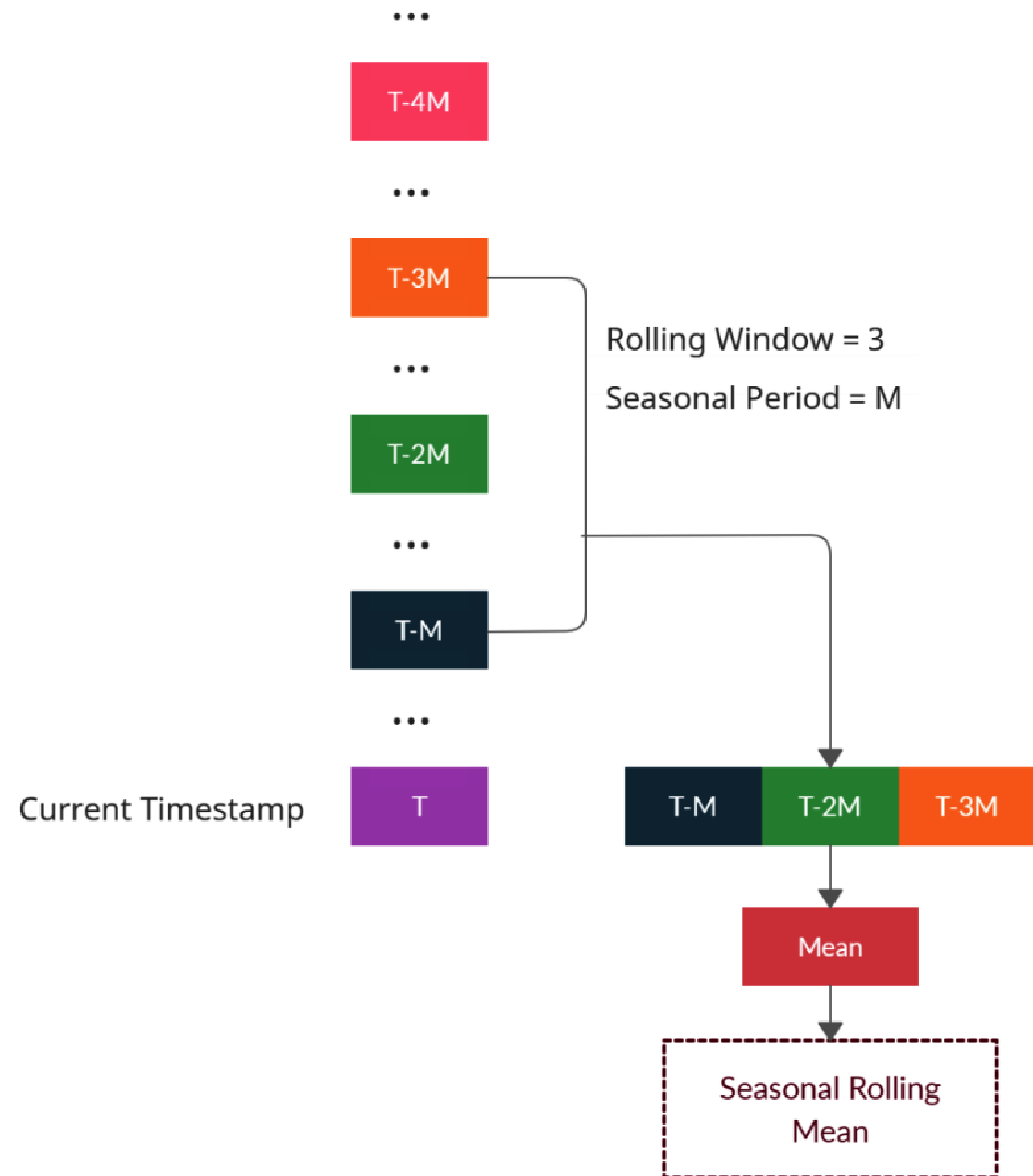
`pandas.DataFrame.rolling`

Seasonal rolling window aggregation

Consider windows with **seasonal observations**.

(Do **not** include x_t in the window! → data **leakage**)

Check the [window-ops](#) package.



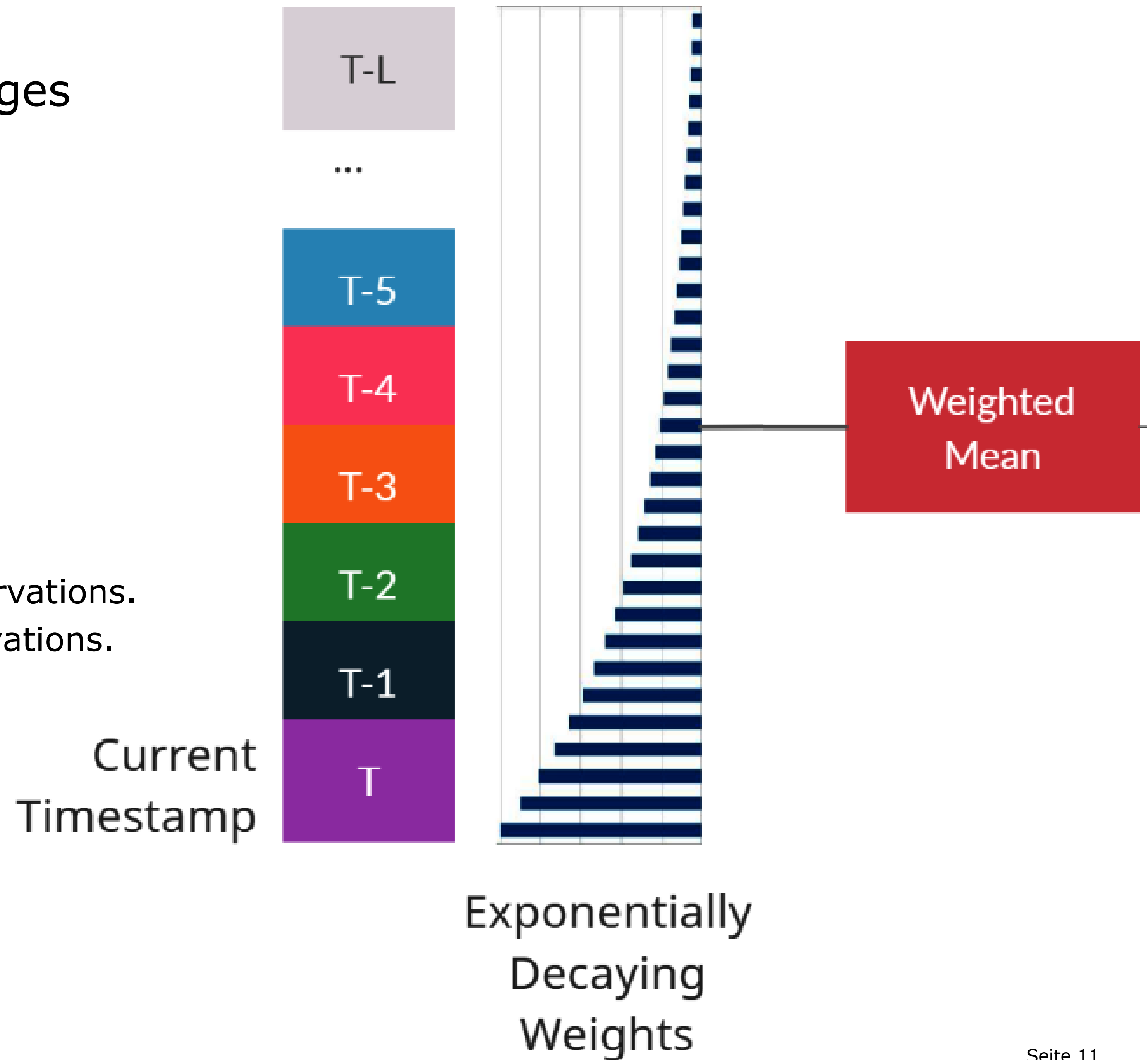
Exponentially weighted moving averages

Recall simple exponential smoothing,
for $\alpha \in [0,1]$ the weight for x_{t-k} is $w_k = \alpha(1 - \alpha)^k$

Consider different values for α

- Large α assigns more weight to **recent** observations.
- Small α assigns more weight to **older** observations.

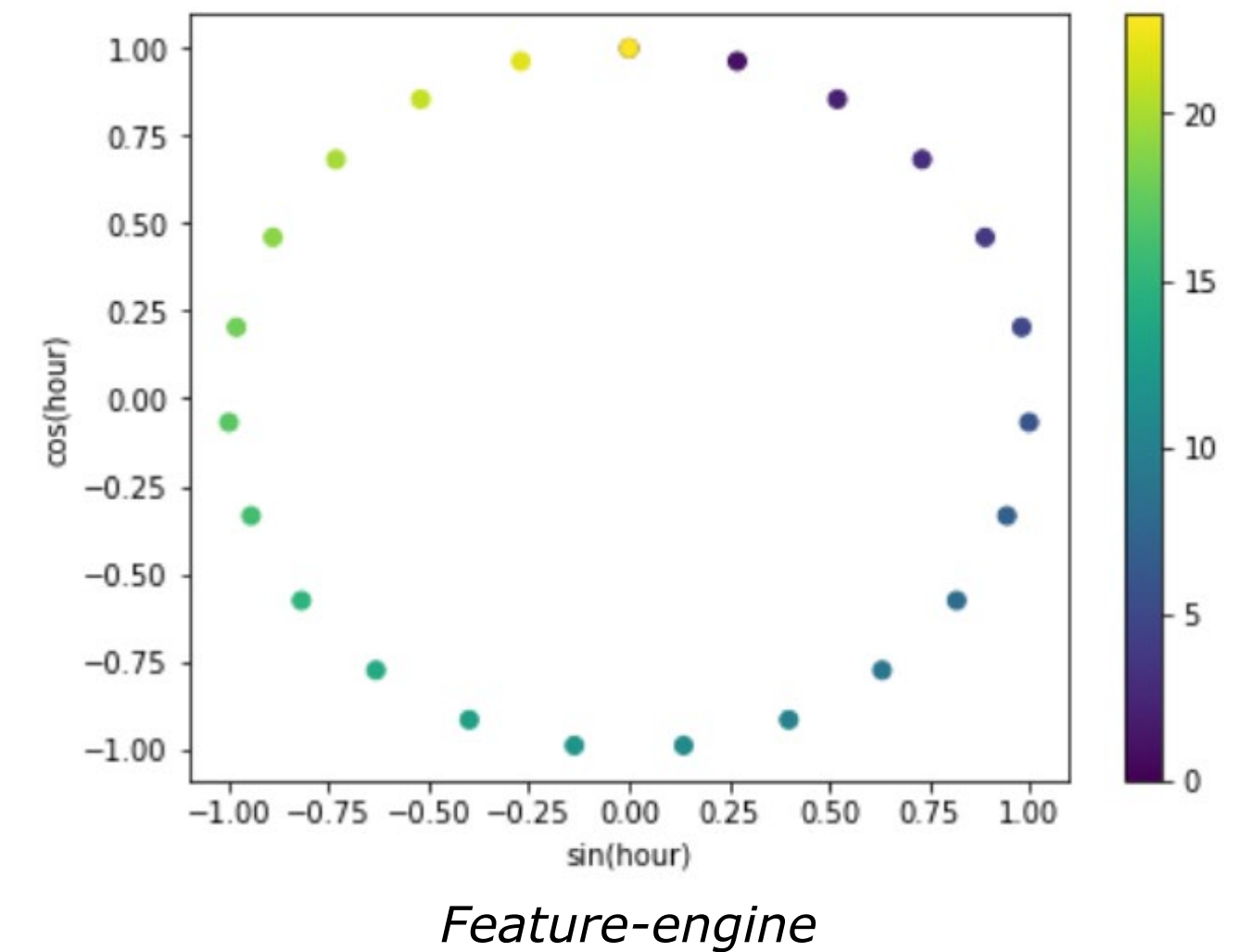
`pandas.DataFrame.ewm`



Temporal features

Calendar features: month, quarter, day, week, hour, ...

- Consider **cyclical encoding**: $x_{1,t} = \sin(x_t \cdot 2\pi / \max(x_t))$, $x_{2,t} = \cos(x_t \cdot 2\pi / \max(x_t))$.



Time elapsed: monotonically increasing feature, giving a sense of the passage of time.

Encode **special events** (e.g., holidays) with dummy variables.

- Spike variable: equal to one for the event of interest and zero elsewhere.
- Step variables: equal to one during the duration of the event (can be permanent e.g., change of trend).
- Beware of the dummy variable trap

Fourier terms for a seasonal period P :

Check `sktime FourierFeatures`.

- $x_{1,t} = \sin(2\pi t/P)$, $x_{2,t} = \cos(2\pi t/P)$, $x_{3,t} = \sin(2\pi t \cdot 2/P)$, $x_{4,t} = \cos(2\pi t \cdot 2/P)$, ...

For more, check *Fulcher, Ben D. "Feature-based time-series analysis.", Feat. eng. for ml and data analytics. 2018.*

Feature importance and selection

Two types of feature importance.

- **Global**: importance over the whole dataset e.g., permutation, leave-one-out.
- **Local**: importance for a single data sample e.g., LIME, SHAP.

Filter methods: statistical methods independent of the model.

- Correlation analysis, statistical tests, mutual information, variance thresholding.
- Efficient but may not capture feature dependencies.

Wrapper methods: evaluate feature subsets by training a model and assessing its performance.

- Recursive feature elimination, forward/backward selection, genetic algorithms.
- Computationally expensive, model-specific, but can lead to better feature sets.

Embedded methods: feature selection integrated in model training process.

- Lasso/Ridge regression, decision trees, bagging/boosting.
- Efficient, model-dependent, requires tuning.

Data preparation – Cleaning



Identification and correction of errors.

- Mistypes, corruption, duplicates, inconsistencies...
- Verify data types (int, float, categorical, etc.)
- Formatting/structure harmonization
- Ensure **consistent** time intervals
- **Granularity** of observations: up/down-sampling

Identification and correction of anomalies & **outliers**.

- Using expert knowledge or statistical analysis: check data ranges, validate plausibility of correlations.

Drop/fill **missing** values (e.g. forward/backward, mean, interpolation), see scikit-learn [imputation of missing values](#).

- Why is the data missing? Is it missing at **random or is there a pattern?**
- How much data is missing, and could imputing it introduce **bias** or distort the analysis?
- Would **removing** the missing data be more appropriate in the context?

Document non-trivial cleaning operations & use dataset versioning e.g., DVC.

Data preparation – Transformations

Adapt data to match ML algorithm assumptions

Encoding variables

- Discretization: numeric (int, floats) to bins: KBinsDiscretizer
- Categorical to numeric: LabelEncoder
- Categorical to ordinal (with rank ordering): OrdinalEncoder
- Categorical to binary vector: OneHotEncoder

Transforming features

- Scaling features: MinMaxScaler, RobustScaler, StandardScaler
- Normalizing data samples to unit norm: Normalizer
- Log transform for skewed data: FunctionTransformer
- Make data more Gaussian: power_transform

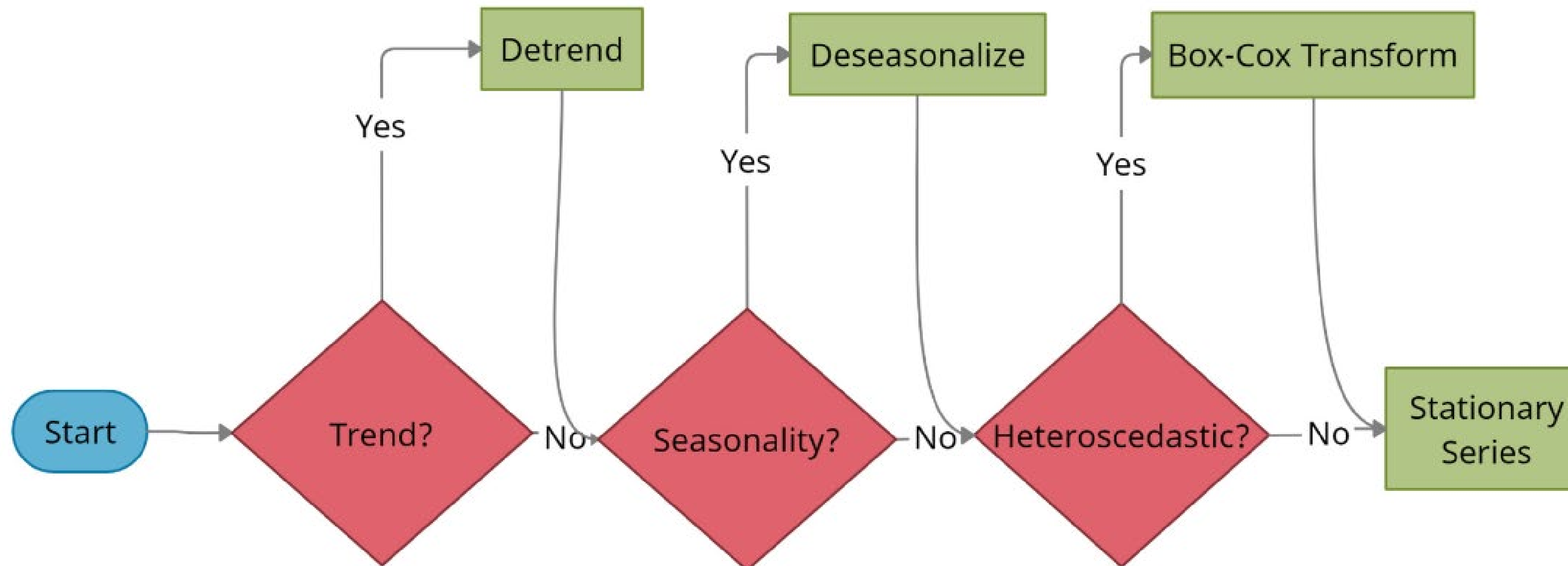
See scikit-learn [preprocessing data](#) and [dataset transformations](#)



Stationarity

Weak stationarity: the **statistical properties** of the time series remain **constant over time**.

Engineering **temporal features can be sufficient** to handle simple forms of non-stationarity. In complex case (or if the model assumes stationarity), making the TS stationary may be necessary.



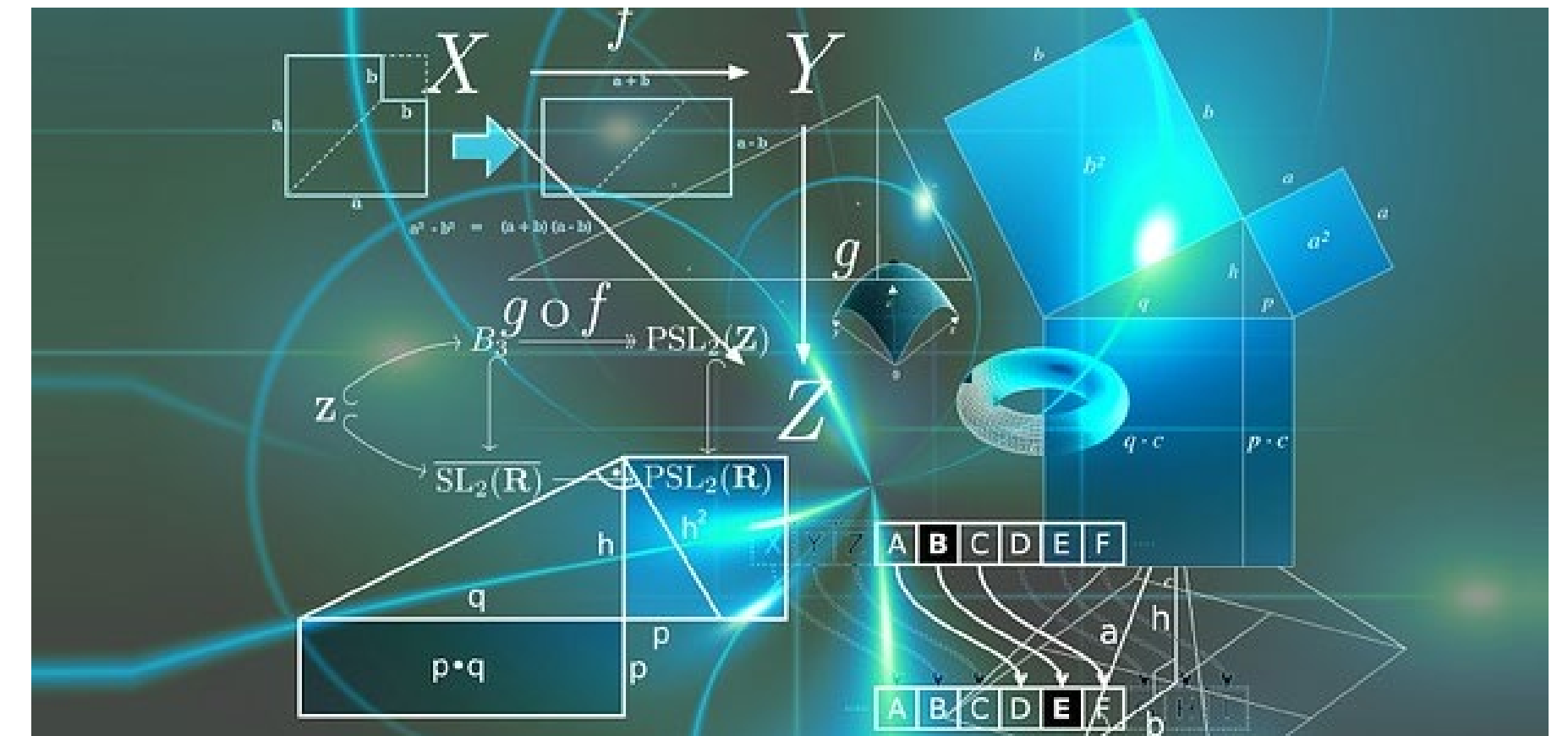
Tour of ML models

The machine learning algorithm zoo is huge

- Instance-based
- Regression
- Decision trees
- Artificial neural networks
- Etc.

Develop intuition/understanding of important algorithms

- Understand assumptions
- Hyperparameters selection
- Debug issues
- Error analysis



Ensemble learning

Get strong predictor from the aggregation of weak predictors

Bagging: weak predictors trained on different random subsets of the training set

- Random forest: collection of decision trees trained from variations of the training set
 - Sampling of data samples with replacement
 - Sampling of data features without replacement

Boosting: train predictors sequentially, each correcting predecessor

- Adaboost: increase weights of misclassified samples
- GradientBoosting: fit new predictors on residual errors

Stacking: meta-predictor trained on weak predictors predictions

- Keep hold-out set to train meta-predictor



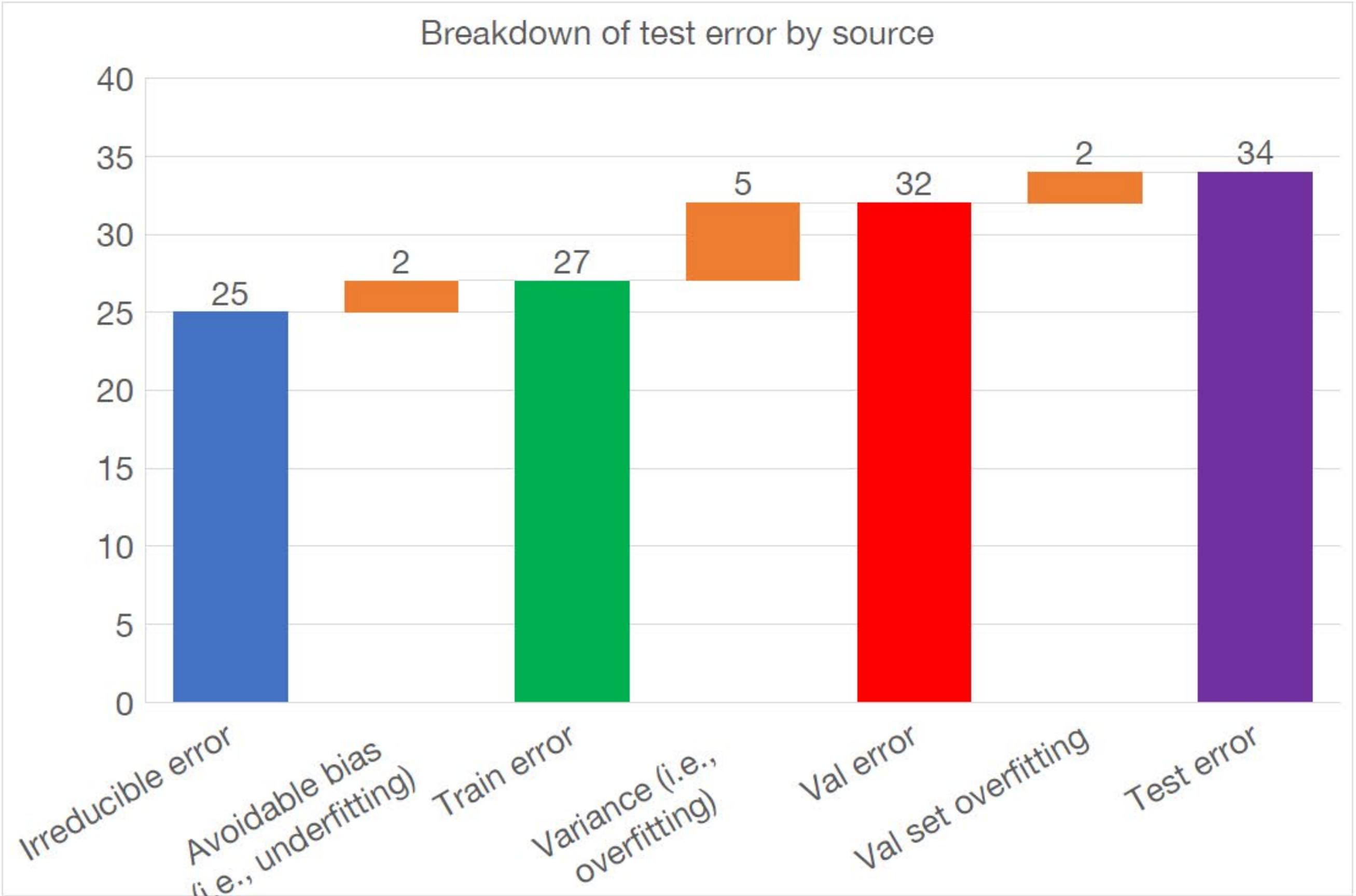
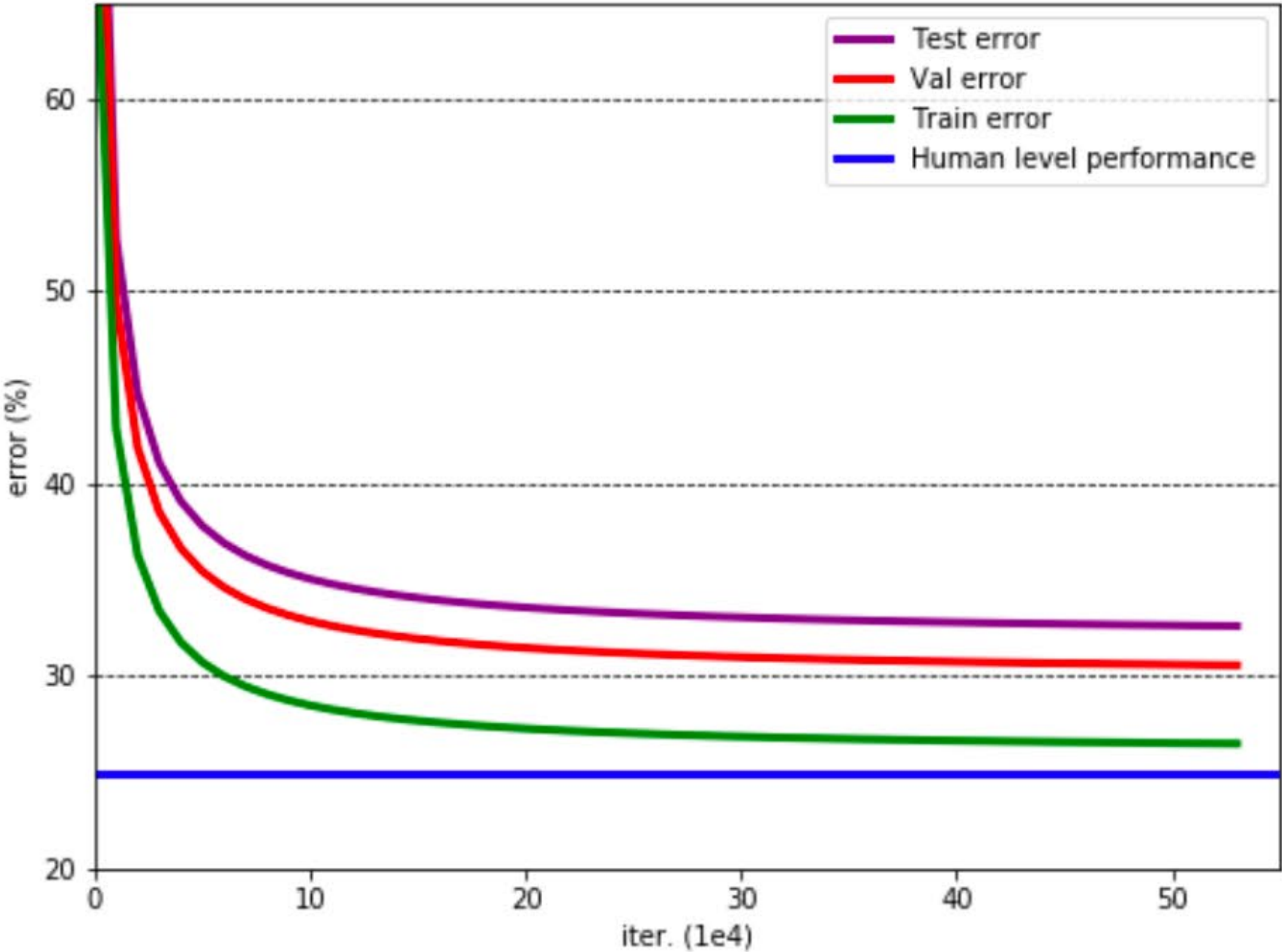
See scikit-learn [ensemble methods](#)

Evaluation workflow

1. **Split dataset** into **training**, **validation**, and **test** sets (splitting method depends on the learning task).
 - Validation set is required either when model fitting involves hyperparameters tuning or when model selection is based on performance metrics.
 - Multiple versions of training and validation sets can be considered with cross-validation.
2. **Train** candidate models on training set.
 - Tune hyperparameters using the validation set.
3. **Select model** based on model fit, complexity and performance on the validation set.
 - Information criteria, residual/error analysis, uncertainty, performance metrics
4. **Train selected model** on training + validation sets then **evaluate performance** on test set.
 - Metrics provide an indication of how well the model will **generalize to new data**.

Error breakdown

Josh Tobin, 2019



Challenges – Limited training data

In practice, only a limited amount of training data is available $\tilde{X} \subseteq X$.

$$\theta^* = \max_{\theta} \sum_{\forall x \in \tilde{X}} P(m_{\theta}(x), t(x))$$

The model parameters θ^* may be optimal on $\tilde{X}$ but suboptimal on $X \setminus \tilde{X}$.

Thus, **$\tilde{X}$ should be as representative as possible of X with the same underlying data distribution.**

How much data is needed?

This depends on complexity of task and chosen algorithm.

- Number of input features (curse of dimensionality).
- Number of target classes (for classification problems).
- Number of model parameters.

Analyze published studies tackling similar problems.

Can be answered only through empirical investigation.

- Plot model performance vs dataset size.

In general, the more data, the better: Halevy, Alon, Peter Norvig, and Fernando Pereira. "The unreasonable effectiveness of data." IEEE intelligent systems 24.2 (2009): 8-12.



Challenges – Bias sources

Data collection

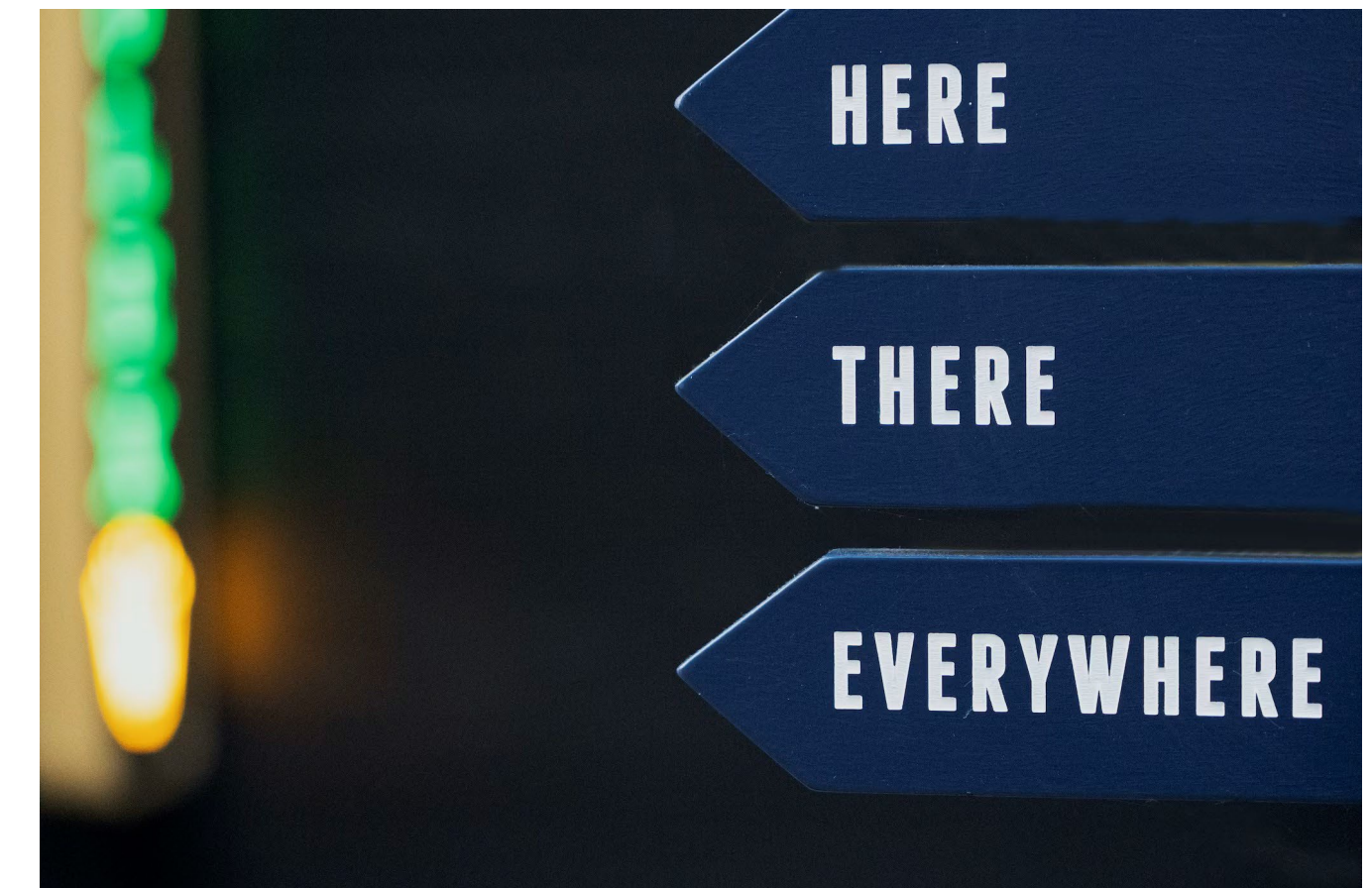
- Capturing process e.g., healthcare access, patient consent
- Inclusion criteria
- Geographic: data collected from limited geographic regions
- Population demographics: age, gender, ...
- Collection centers: specialized centers with different disease prevalence

Annotation

- Variability in the expertise level.
- Intra and inter-observer variations

Evolution over time

- Seasonal bias
- Data drift



Data probability distribution must be/remain consistent between training and inference.

Causality

- In causal setting, i.e. X causes Y, covariate shift and concept shift are considered.
- In anti-causal setting, i.e. Y causes X, label shift and manifestation shift are considered.
- Covariate and label shifts are indistinguishable when causality is unknown.

Differences in data distribution can introduce bias and degrade model performance.

Type of shift	Data $P(X)$	Targets $P(Y)$	Concept $P(Y X)$	Manifestation $P(X Y)$
Covariate shift	Change	Change	Same	Same
Concept shift	Same	Change	Change	Same
Label shift	Change	Change	Same	Same
Manifestation shift	Change	Same	Same	Change

$$P(Y, X) = P(Y|X)P(X) = P(X|Y)P(Y) = P(X, Y)$$

Challenges – Imbalanced dataset

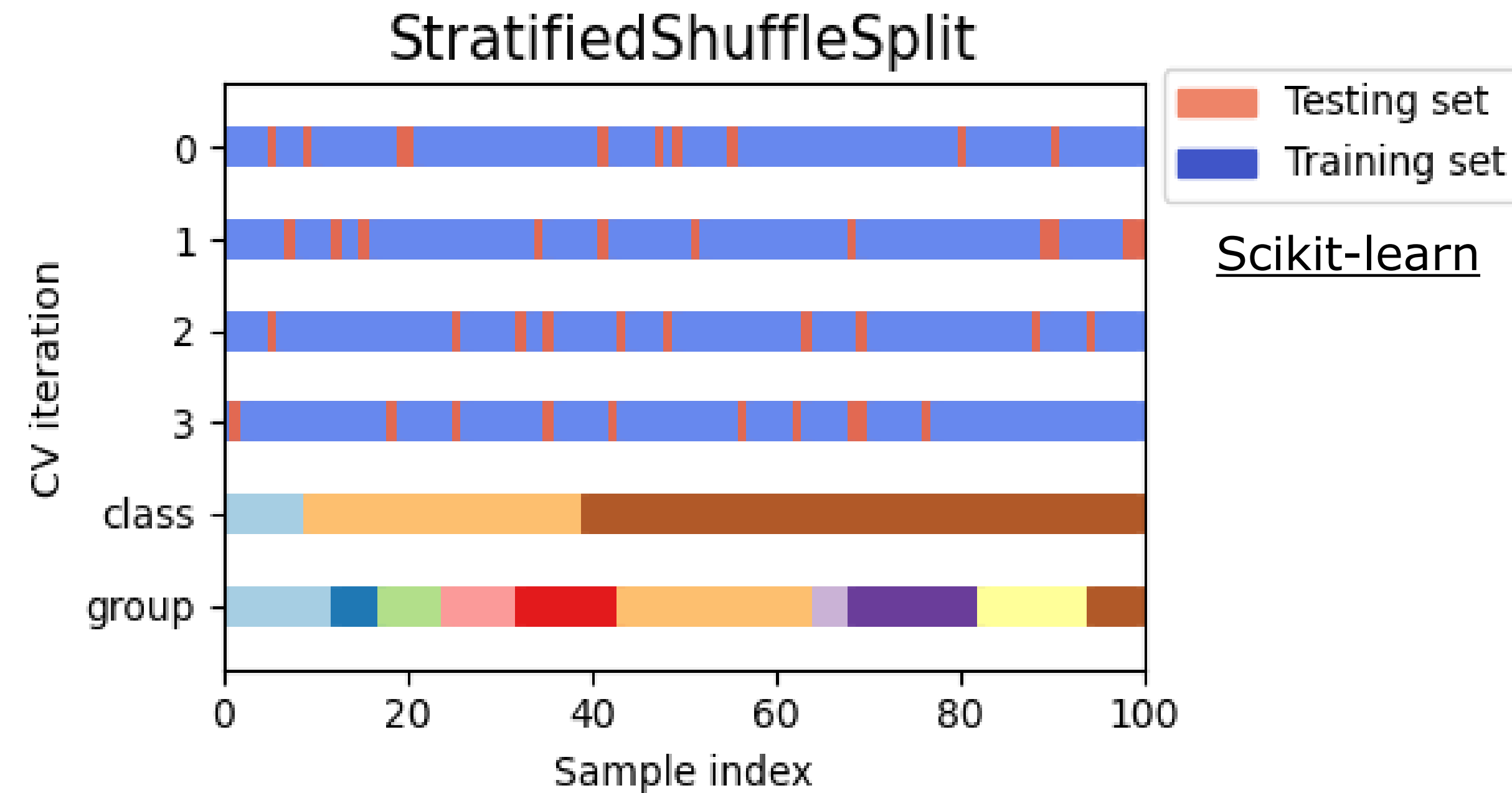
Imbalance in **classification** tasks.

Use **stratified** splits to preserve target distribution.

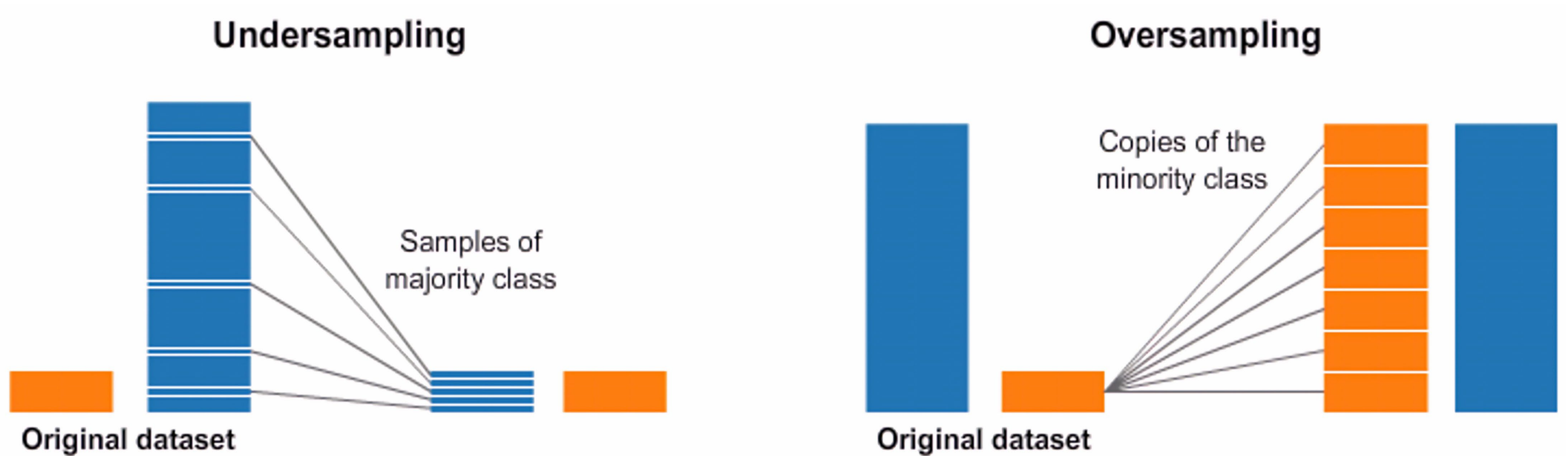
Merge **minority classes**.

Use appropriate metrics: balanced accuracy, precision, recall, F1-score, area under Precision/Recall curve.

Here, each sample represents a whole time series.



Imbalanced dataset – Over/under-sampling



Liu et al. 2019

Final proportions do **not** have to be equal !

Exercise

Review Kaggle tutorials:

- [Machine learning](#)
- [Intermediate machine learning](#)
- [Time series](#)

Model real-world time series.

- Engineer temporal features.
- Train time series models.
- Make data stationary and retrain models.
- Compare with classical time series models such as ARIMA and ETS.

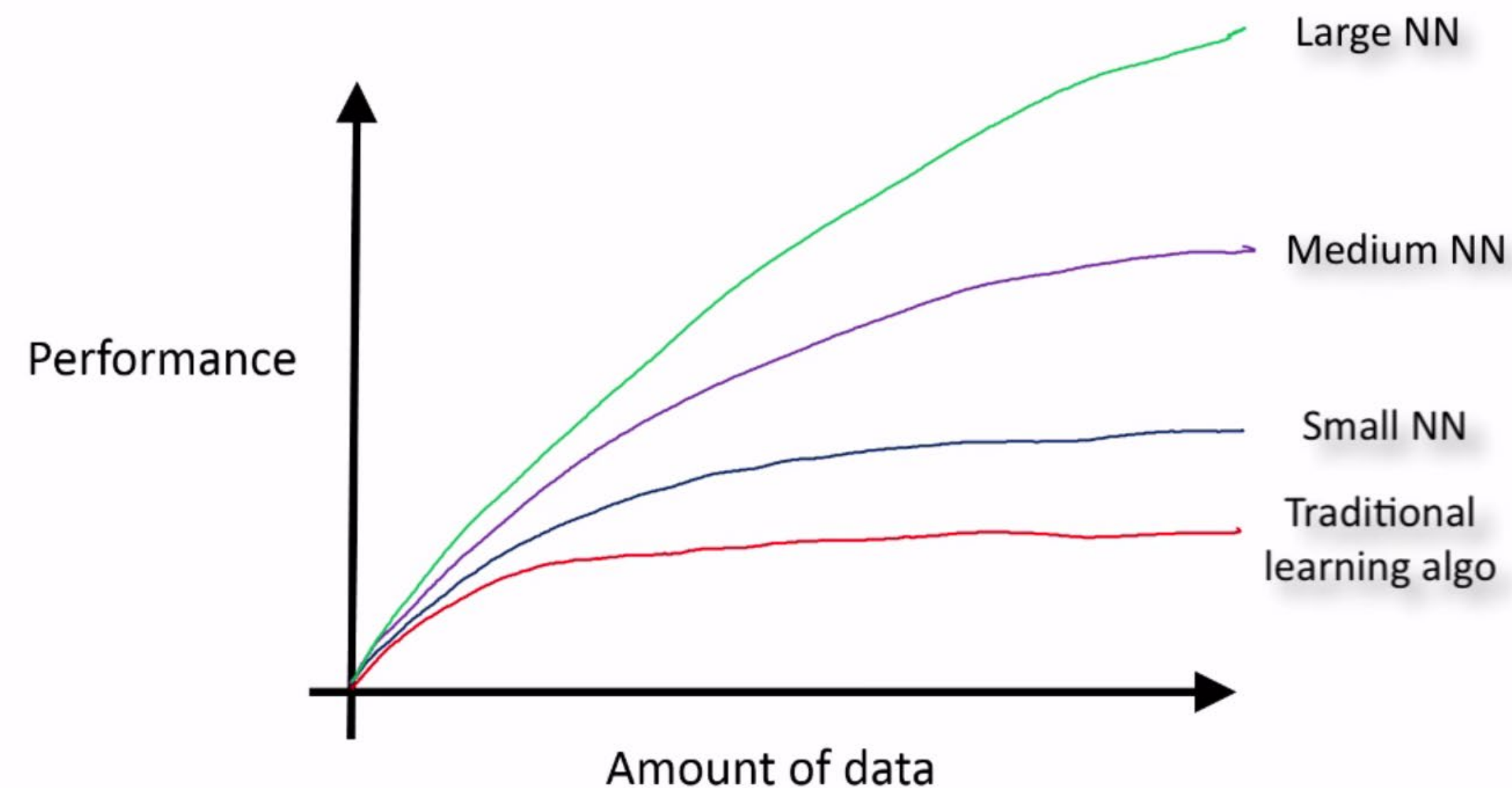
Deep learning (1)

Sub-field of machine learning where algorithms are

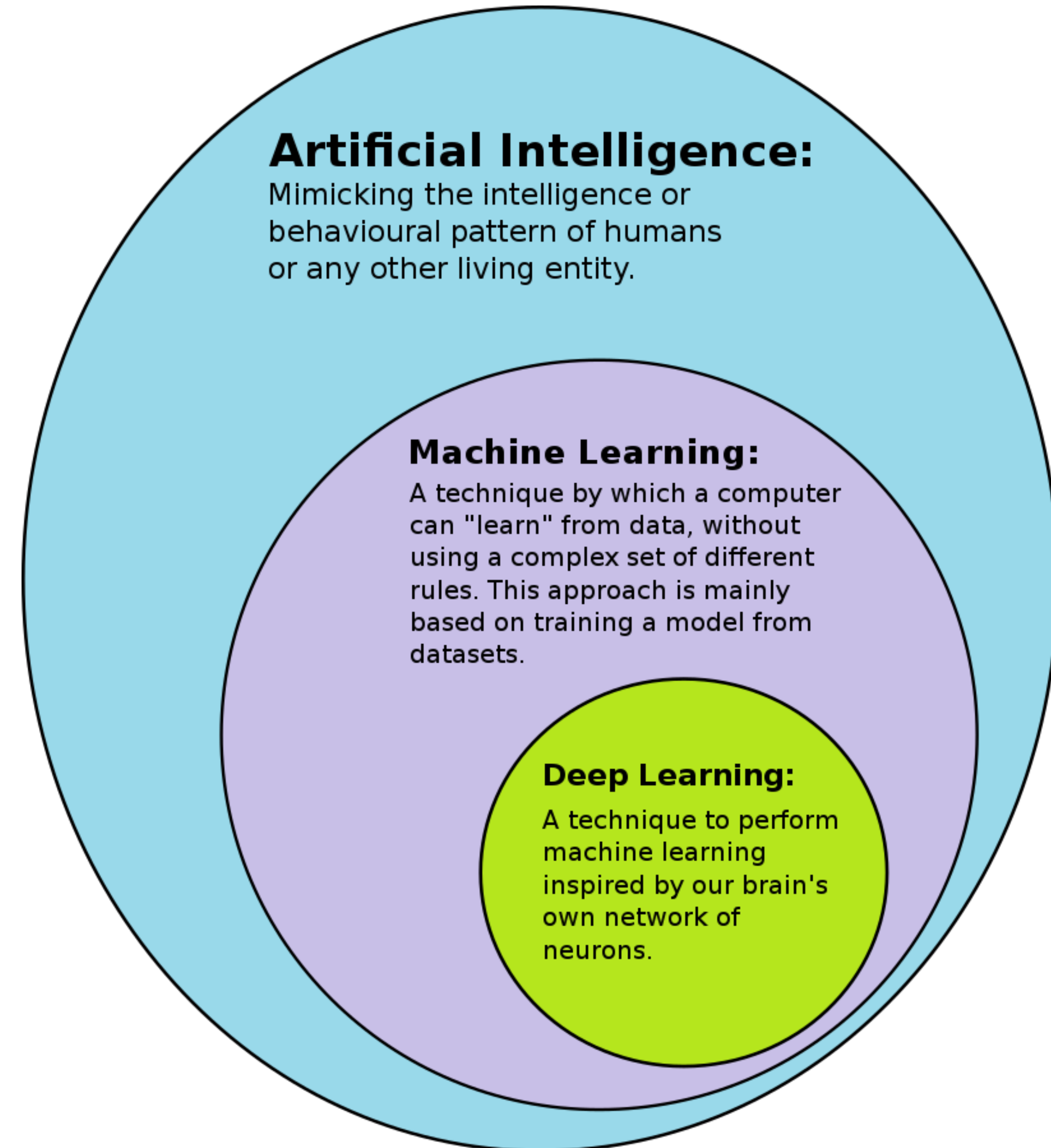
- Composed of **parametrized modules**
- Optimized with **gradient-based methods**.

While traditional ML relies on **hand-crafted features**,

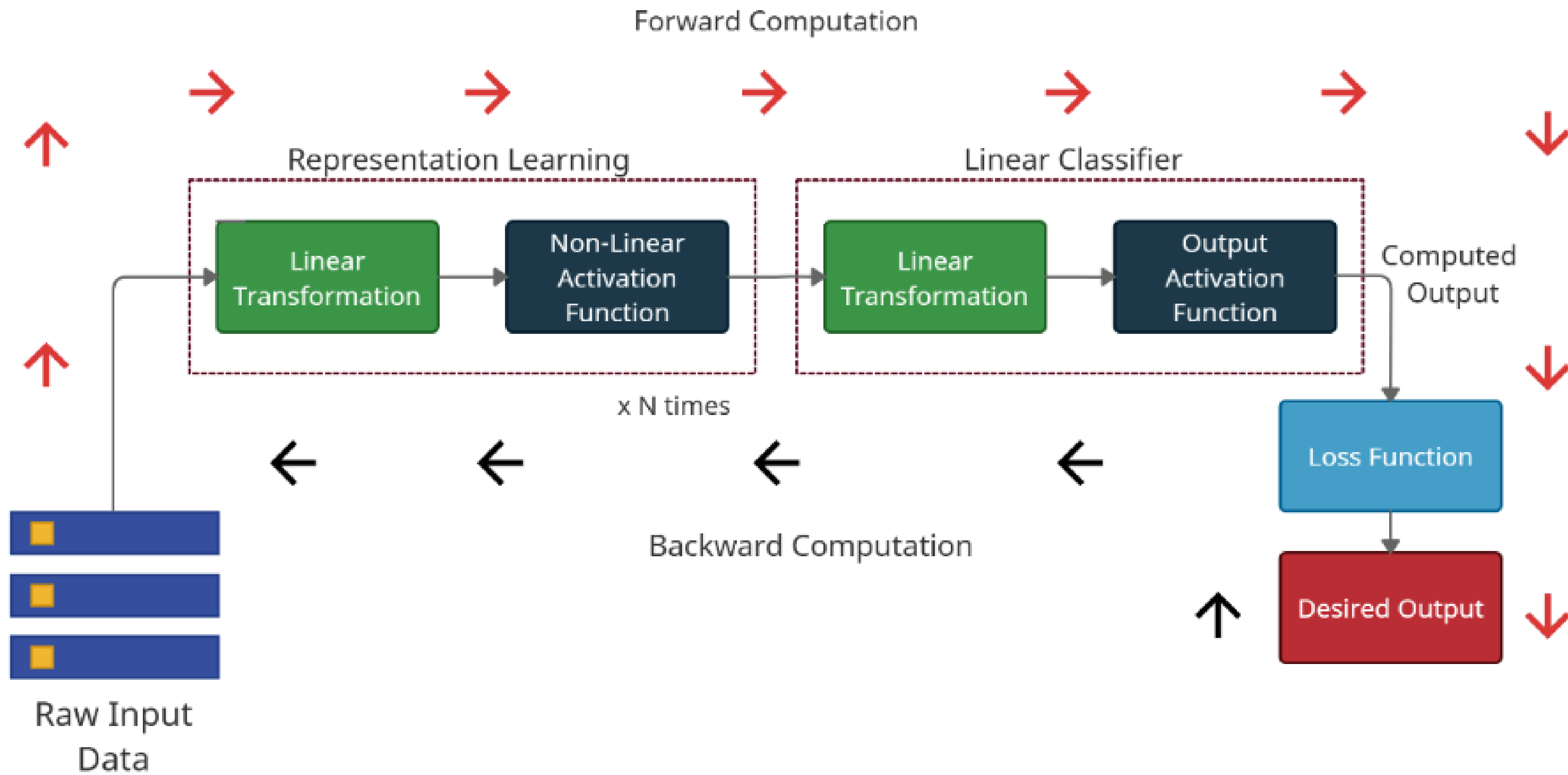
DL autonomously learns hierarchies of features from **(un)structured data**: text, audio, images, time series ...



Andrew Ng

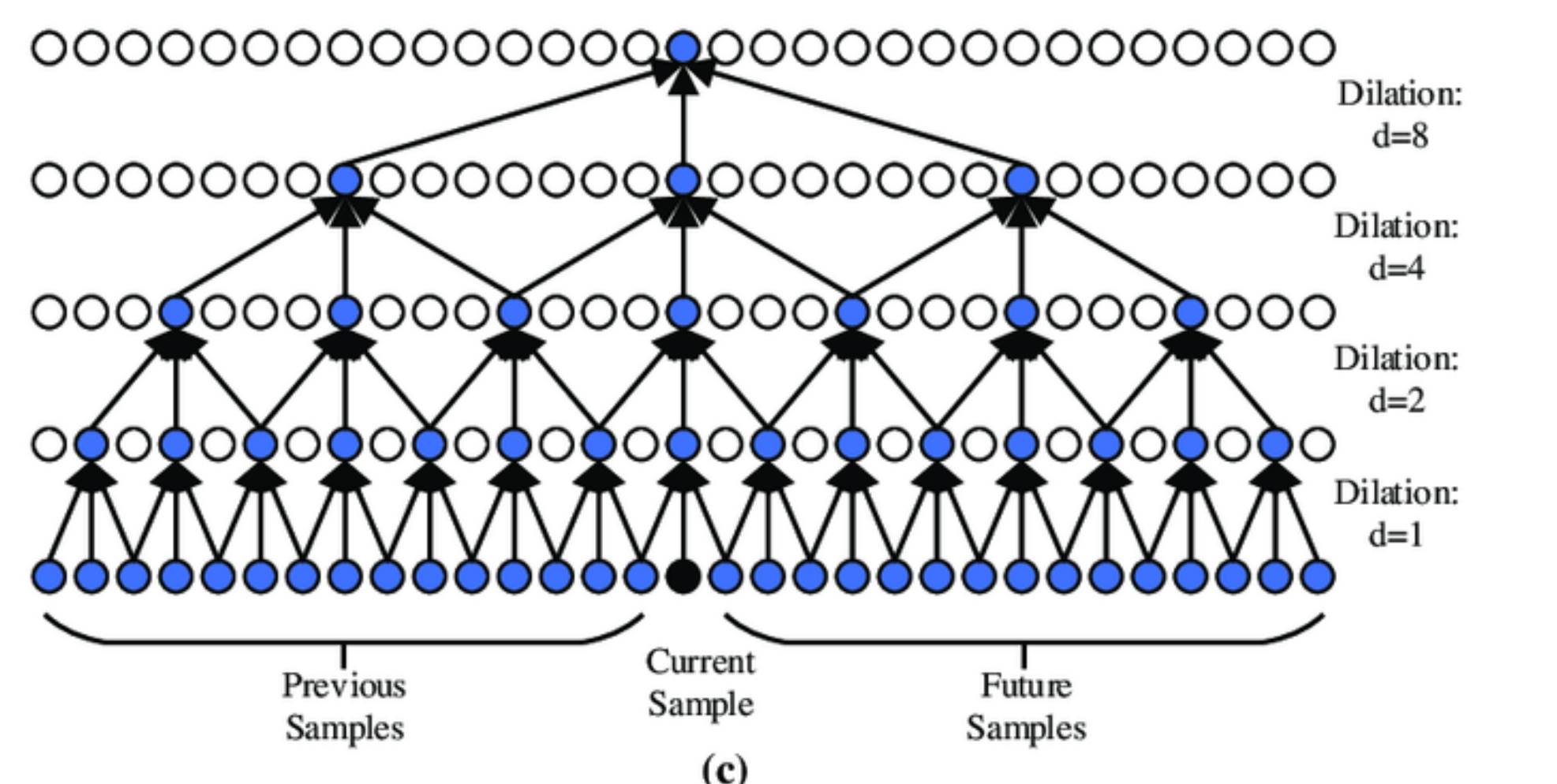
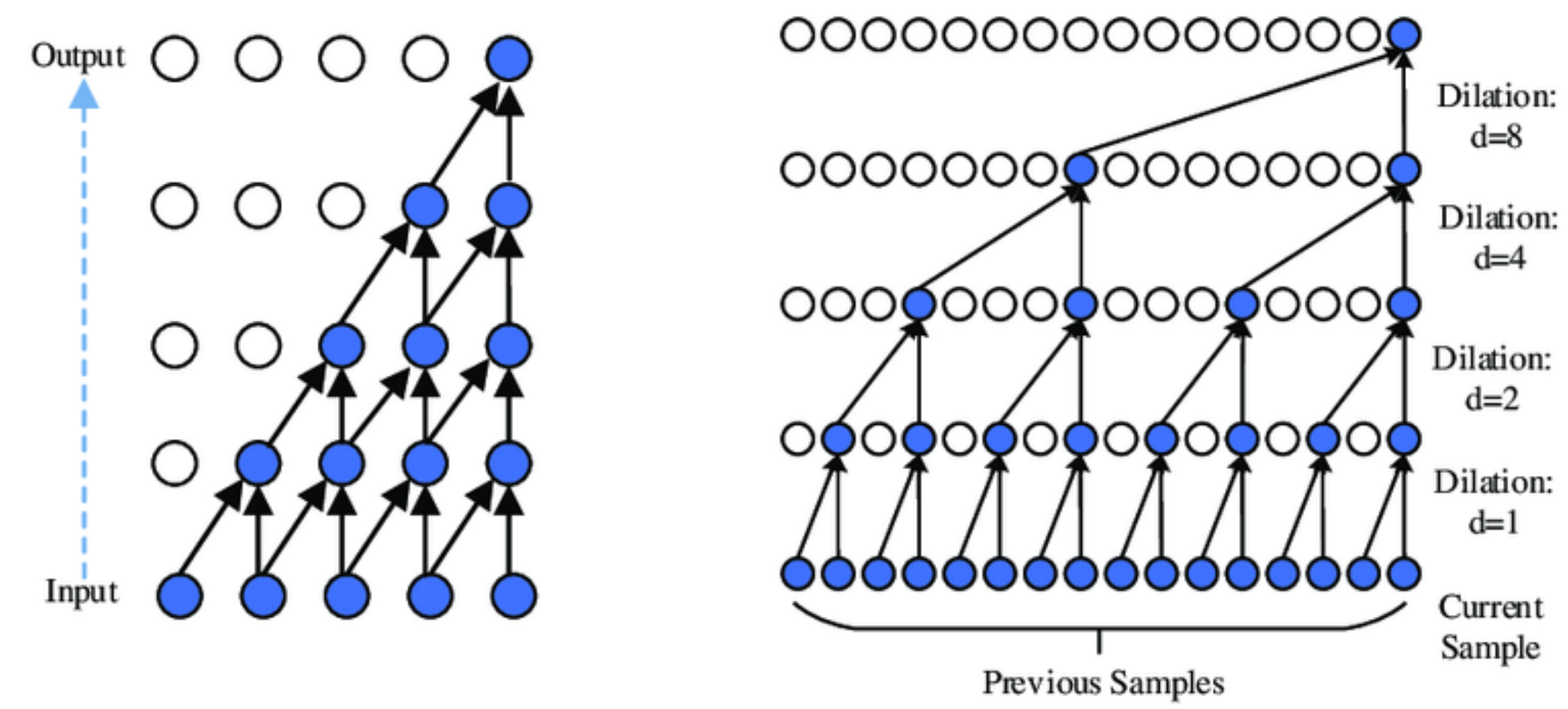
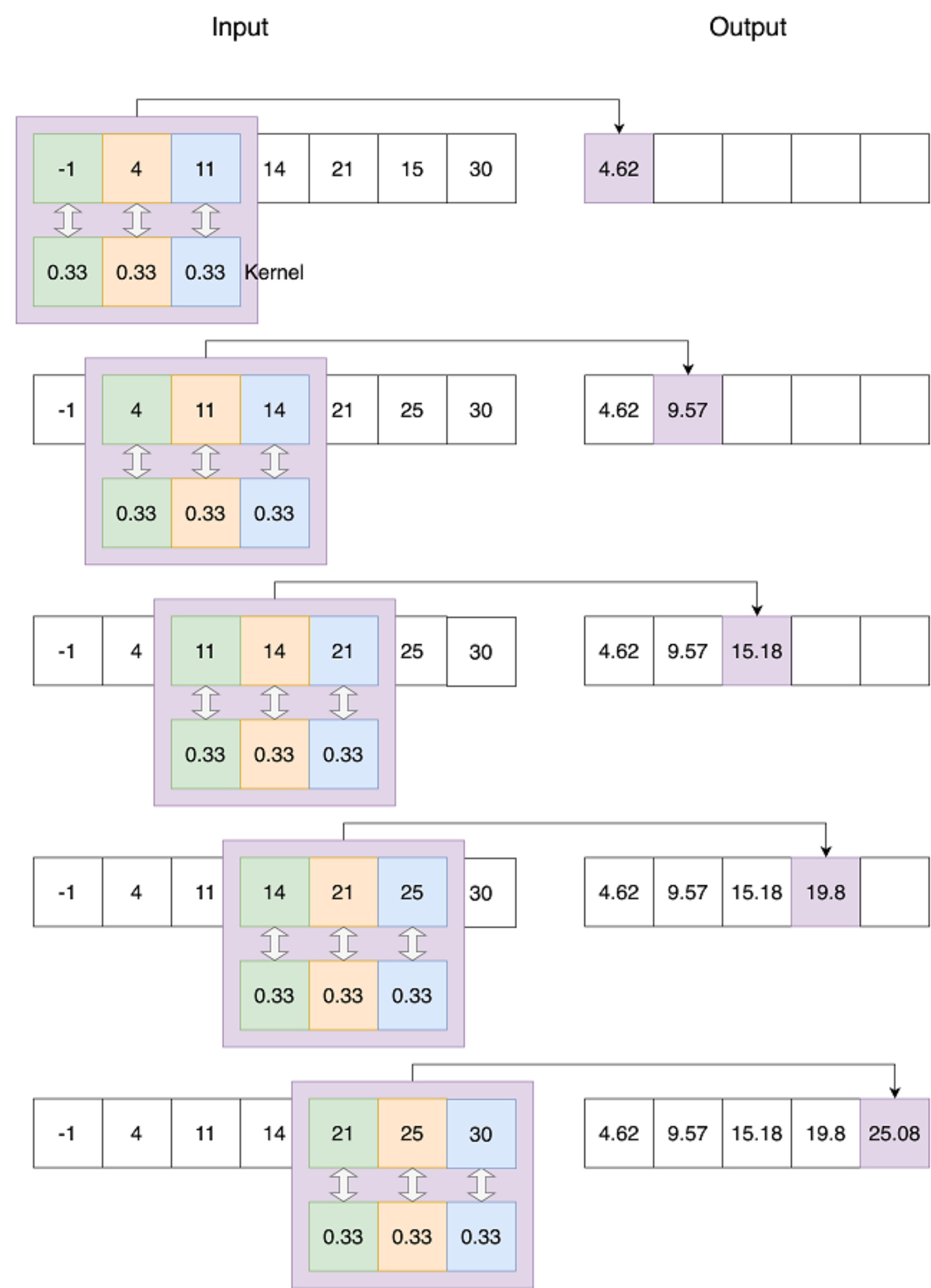


Deep learning (2)



1D Convolution

Stride = 1
Kernel size = 3

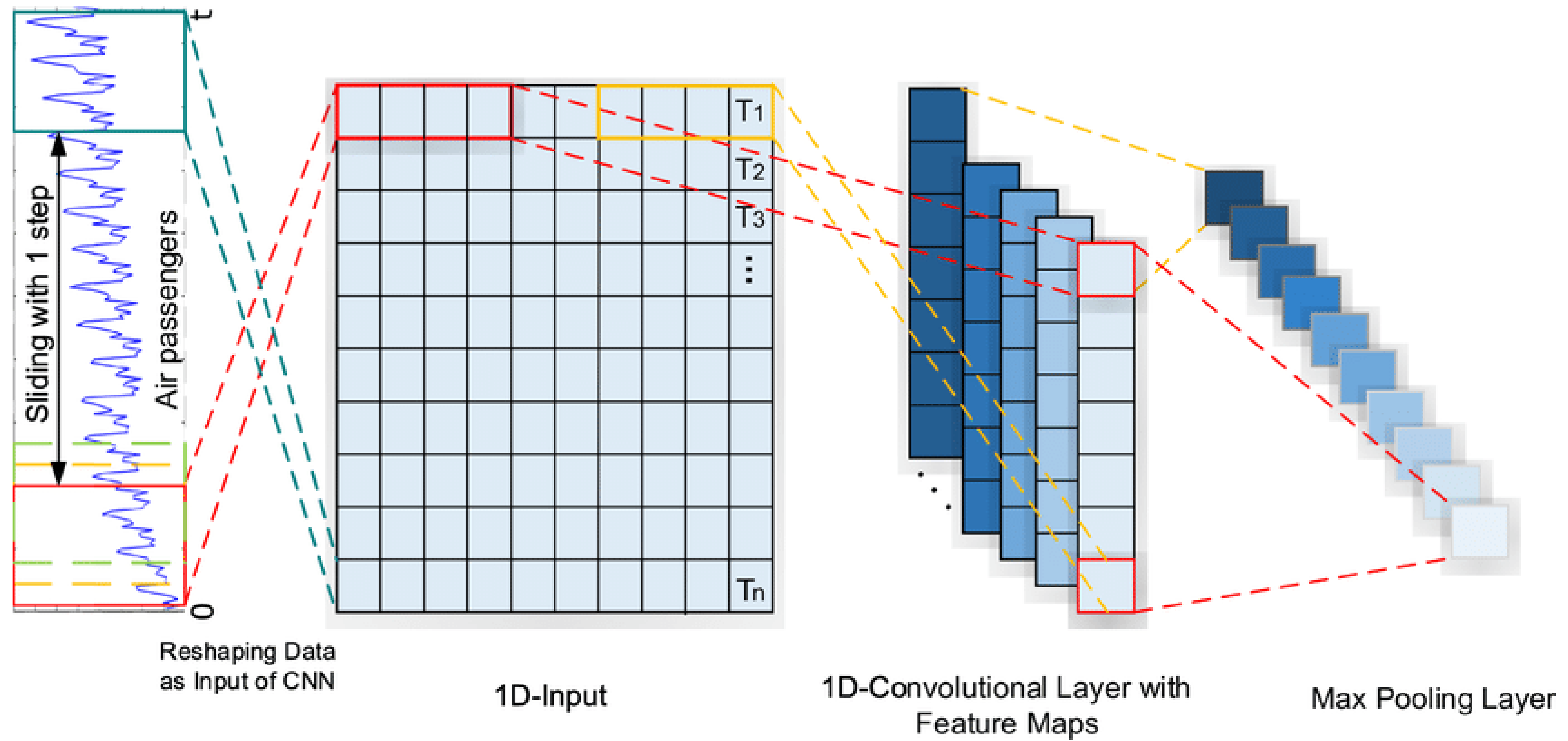


Shu, Yifei, et al. "Load Disaggregation Based on a Bidirectional Dilated Residual Network with Multihead Attention." *Electronics* 12.12 (2023): 2736.

Causal convolution: ensure that no future time steps is used to predict the current timestep.

Receptive field: the region in the input associated with a feature.

Convolutional neural networks (CNN)



Recurrent neural network (RNN)

RNN are designed to handle **sequential data**.

RNN maps the **input sequence** $\{x_1, \dots, x_n\}$ to an **output sequence** $\{o_1, \dots, o_n\}$ based on a **hidden state sequence** $\{h_0, h_1, \dots, h_n\}$.

- h_0 is typically initialized with zeros.

At each time step,

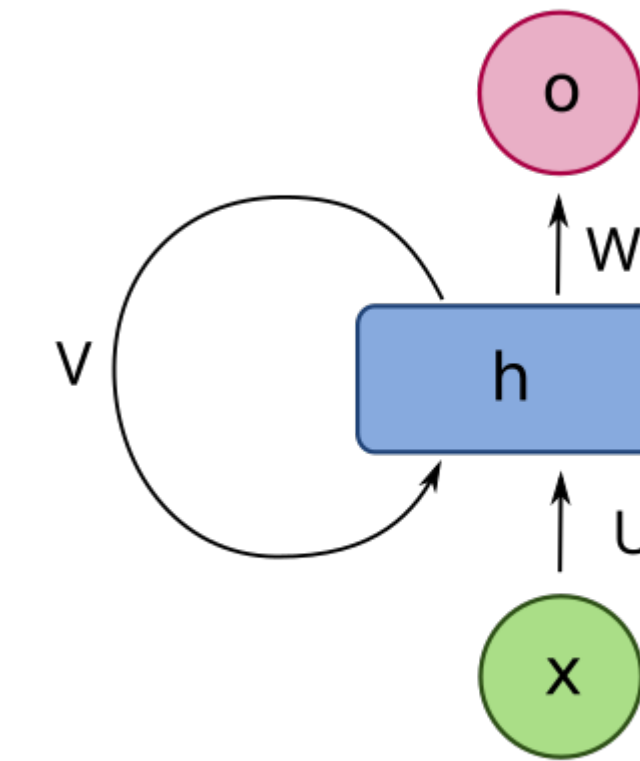
$$h_t = \phi_h(Ux_t + Vh_{t-1} + b_h)$$

$$o_t = \phi_o(Wh_t + b_o)$$

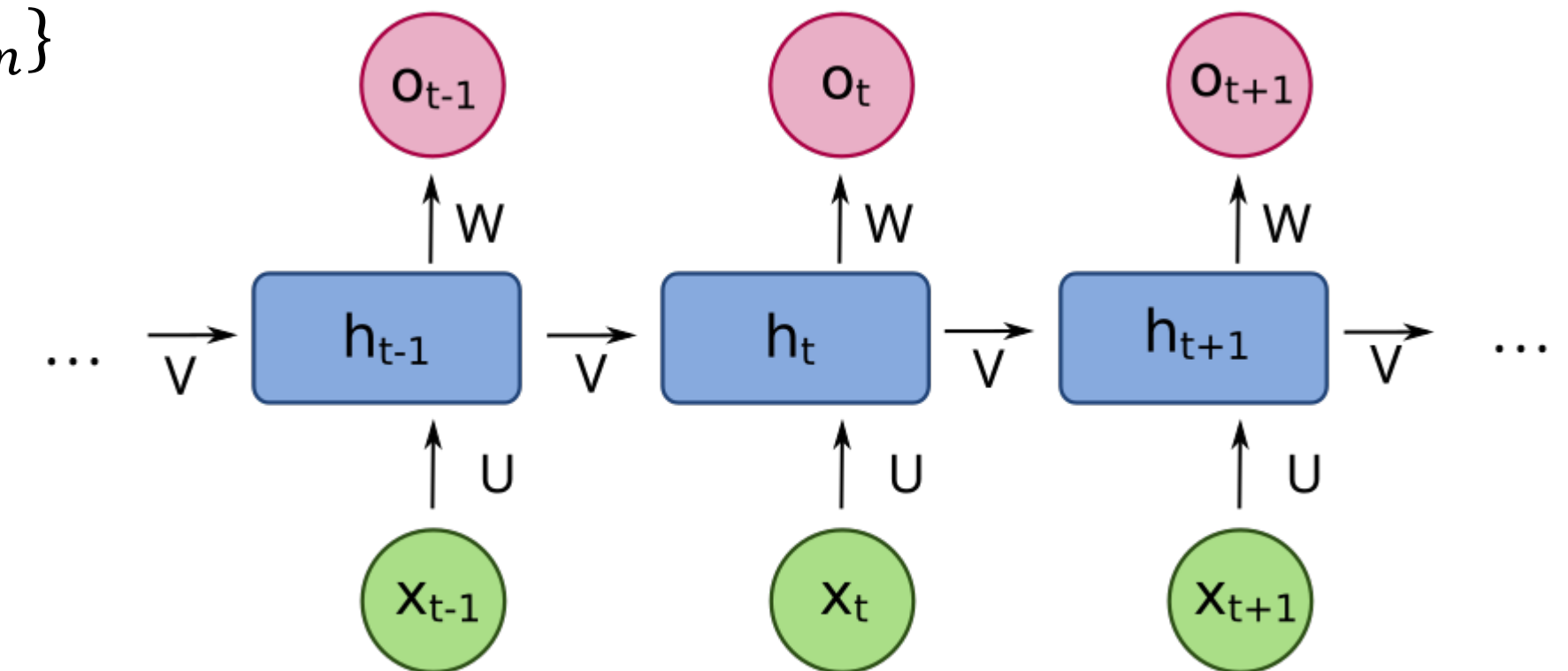
with U, V, W **learnable** weight matrices and b_h, b_o **learnable** bias vectors.

Backpropagate gradients within single unit through time steps.

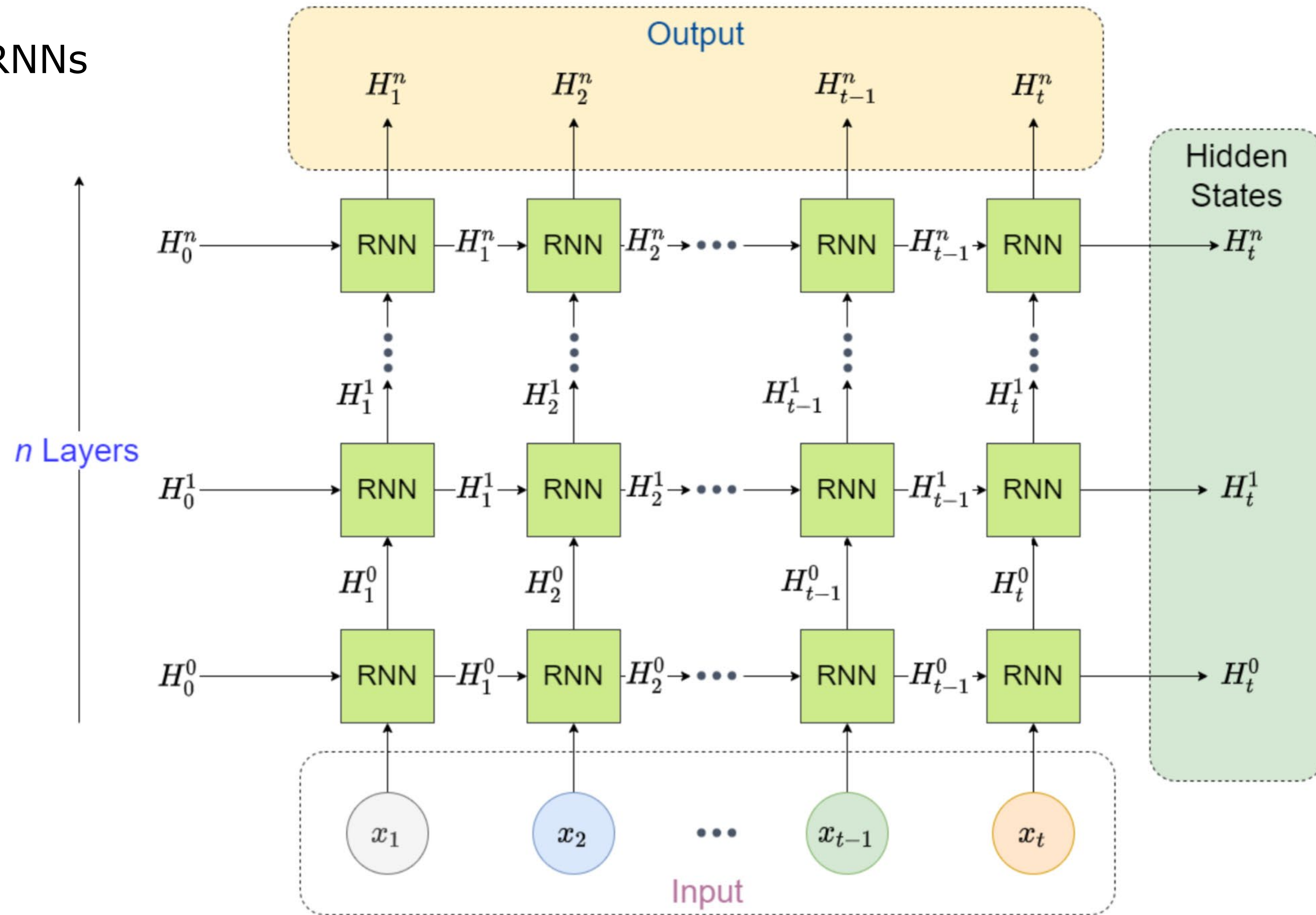
- **Vanishing/exploding gradient** problem with long sequences.



Fdeloche, wikimedia



Stacking RNNs



Long Short-Term Memory (LSTM)

Fdeloche, wikimedia

Forget gate: what information can be discarded.

$$F_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

Input gate: what information should be added.

$$I_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

Candidate memory cell state:

$$\tilde{c}_t = \tanh(W_{\tilde{c}}[h_{t-1}, x_t] + b_{\tilde{c}})$$

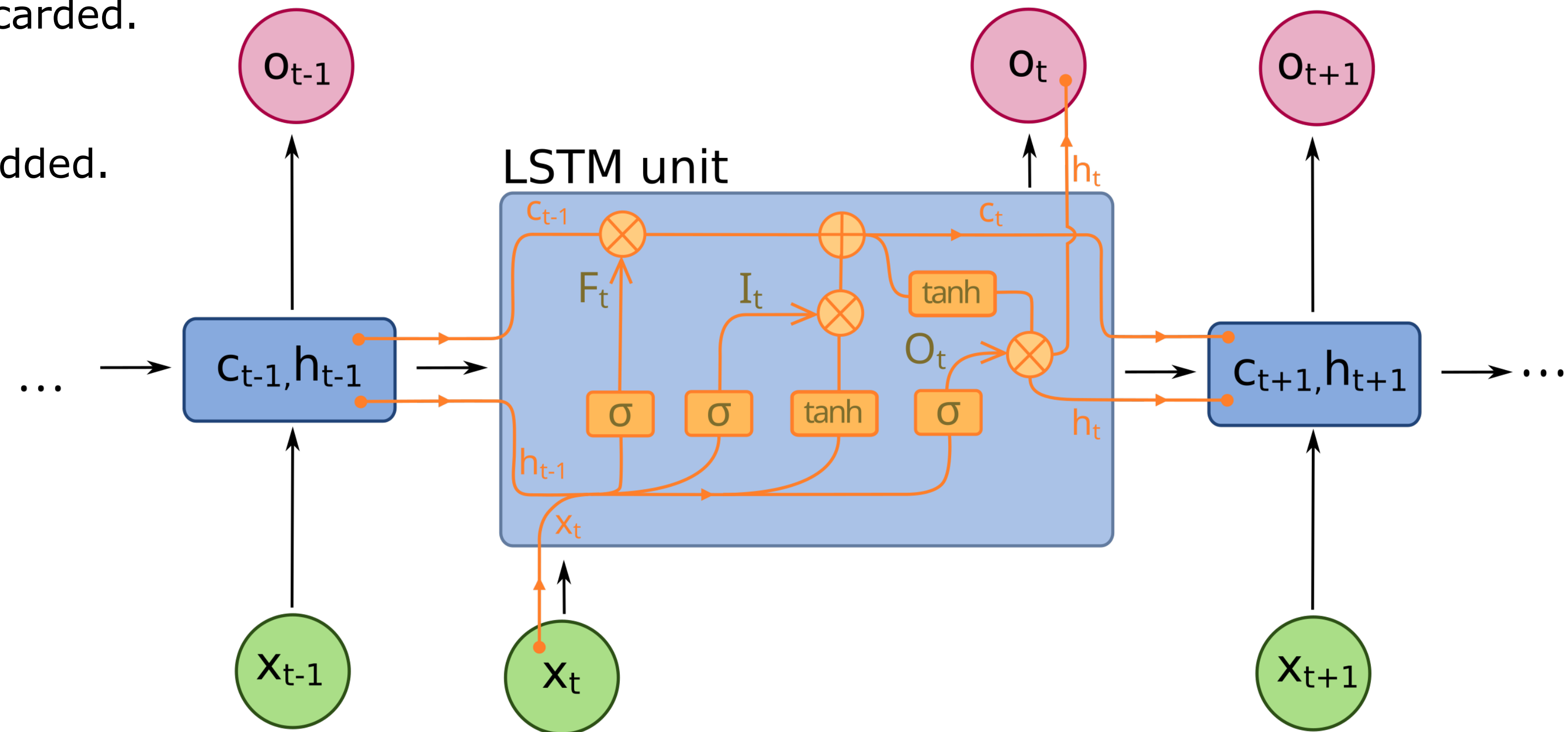
Output gate:

$$O_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

Cell state:

$$c_t = F_t \otimes c_{t-1} + I_t \otimes \tilde{c}_t$$

$$h_t = O_t \otimes \tanh(c_t)$$



The cell output is its state h_t . The memory cell c_t serves as long-term memory and “gradient highway”.

$W_{...}$ are **learnable** weight matrices and $b_{...}$ **learnable** bias vectors.

$\otimes$ is elementwise multiplication.

σ is the sigmoid activation.

$[h_{t-1}, x_t]$ represents concatenation.

Gated recurrent unit (GRU)

Fdeloche, wikimedia

Simplification of LSTM:

- No memory cell.
- Use hidden state h_t as “gradient highway”.

Reset gate:

$$R_t = \sigma(W_r[h_{t-1}, x_t] + b_r)$$

Update gate:

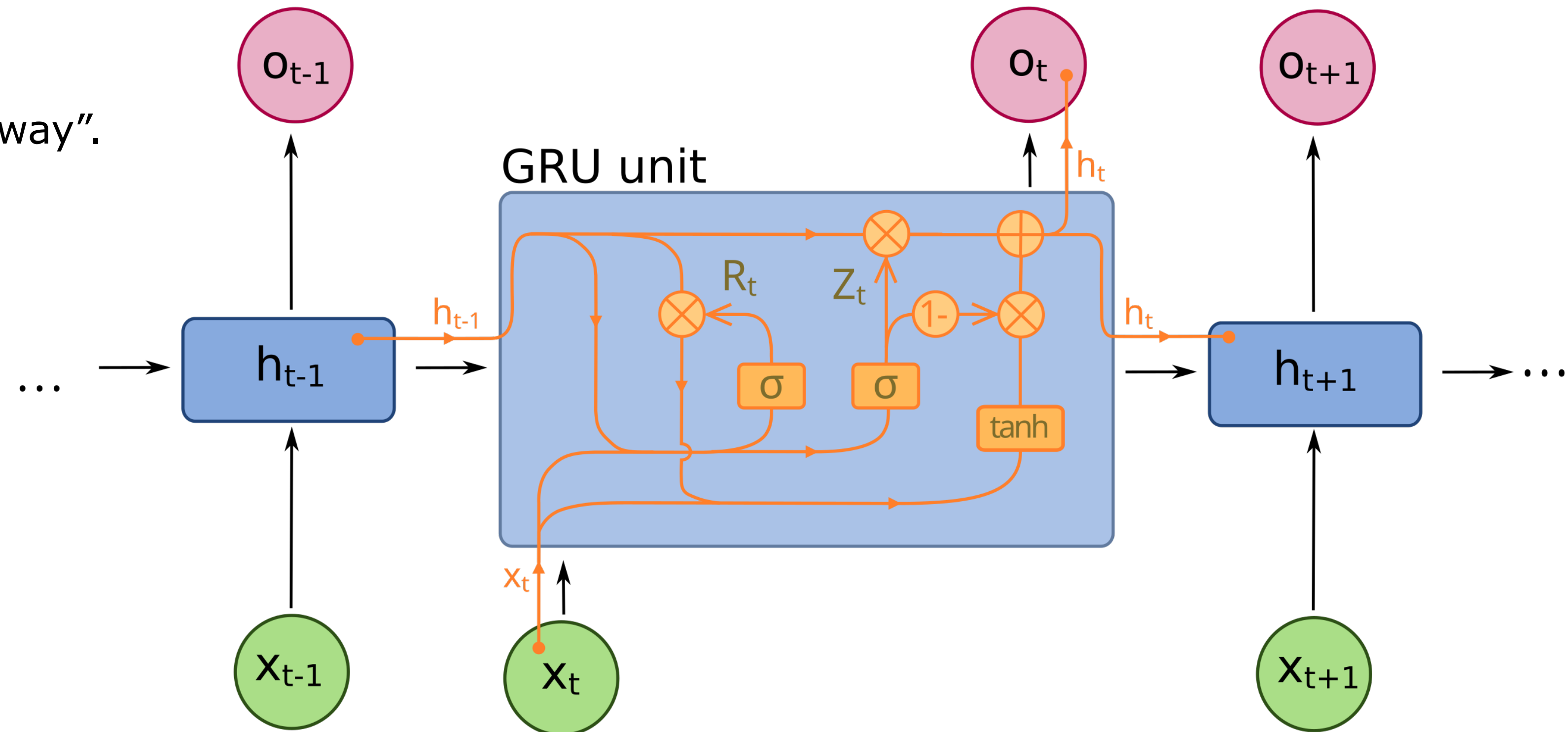
$$Z_t = \sigma(W_z[h_{t-1}, x_t] + b_z)$$

Candidate hidden state:

$$\tilde{h}_t = \tanh(W_{\tilde{h}}[R_t \otimes h_{t-1}, x_t] + b_{\tilde{h}})$$

Cell state:

$$h_t = Z_t \otimes h_{t-1} + (1 - Z_t) \otimes \tilde{h}_t$$



$\otimes$ is elementwise multiplication.

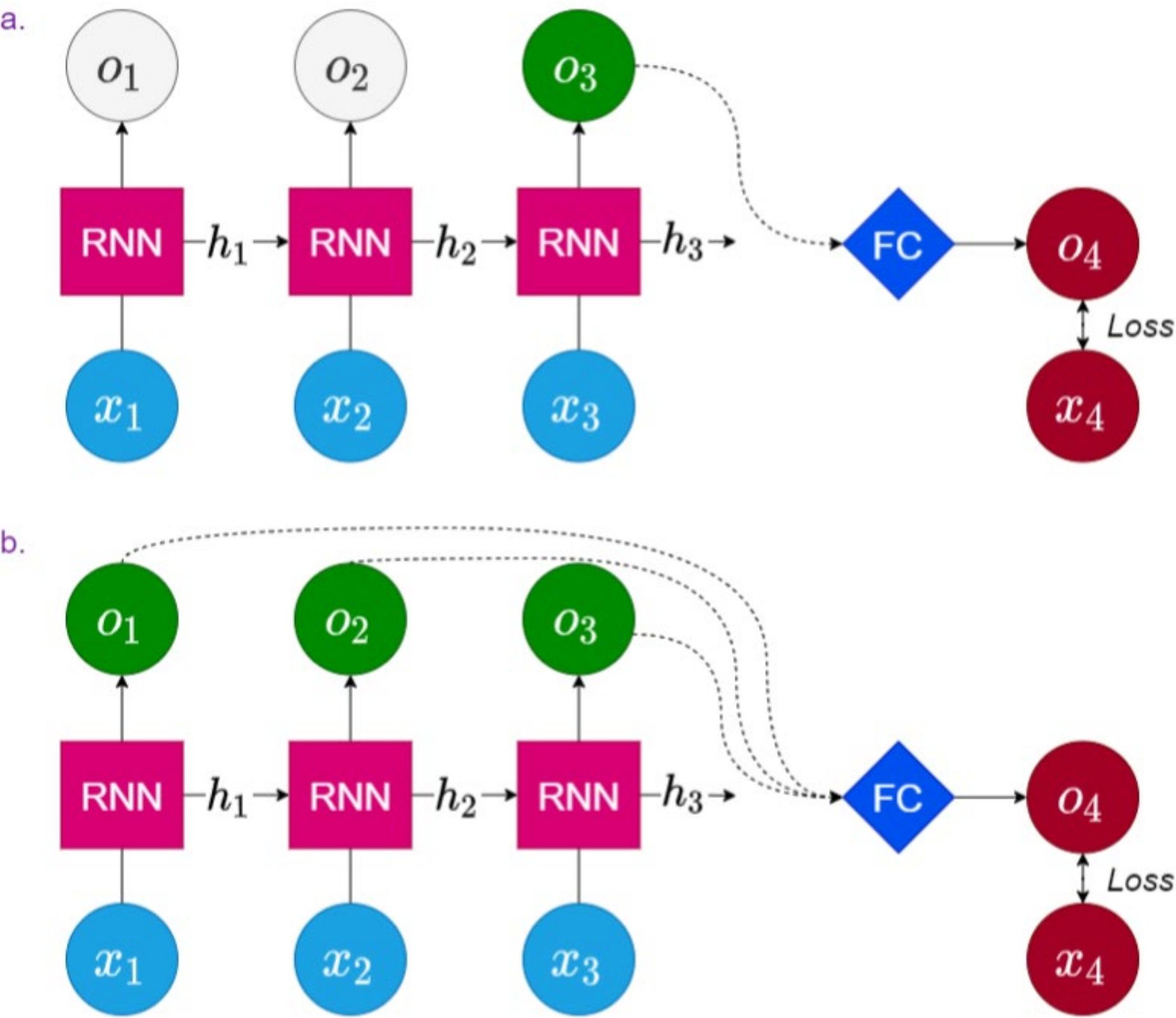
σ is the sigmoid activation.

$[h_{t-1}, x_t]$ represents concatenation.

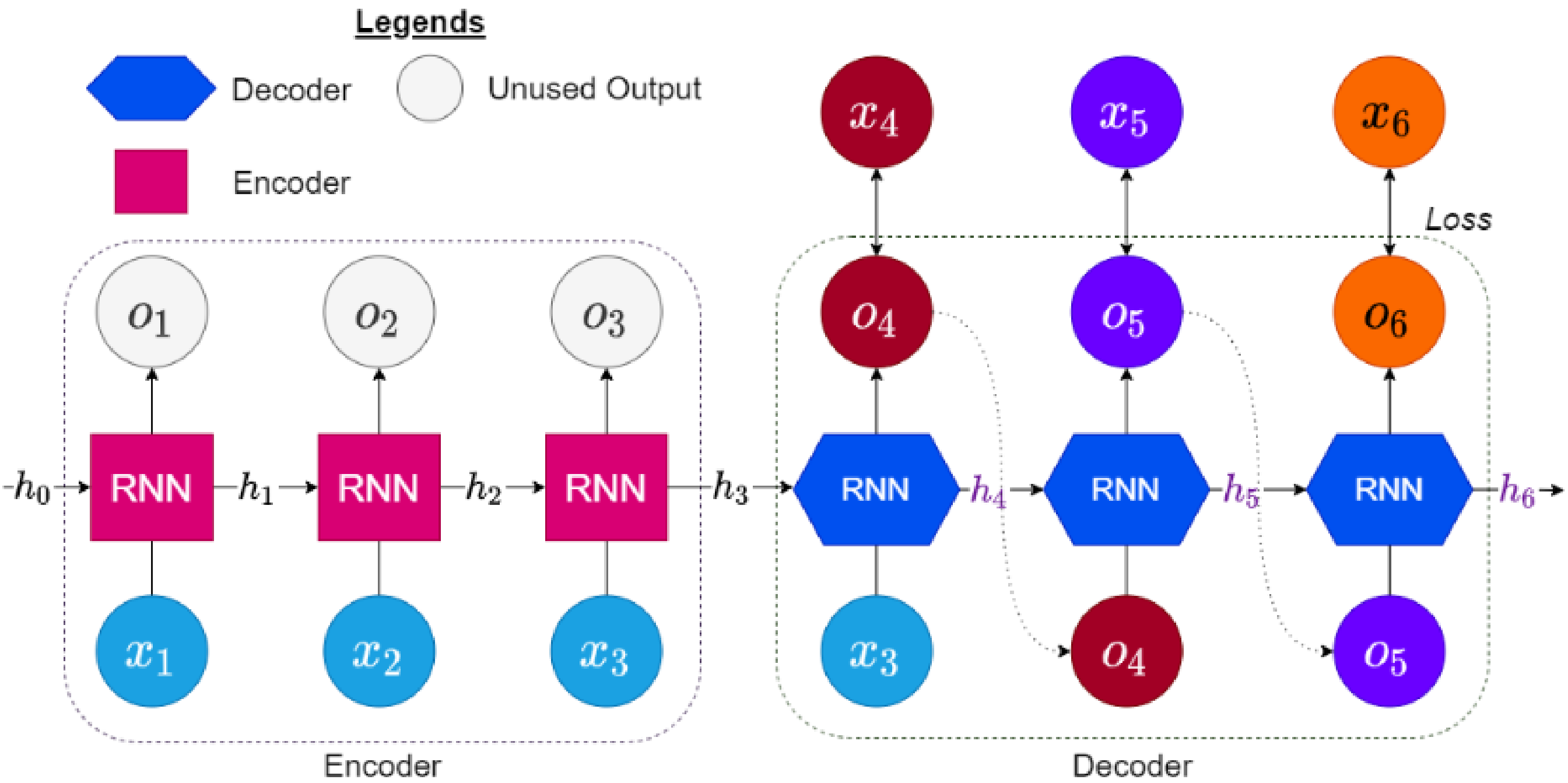
$W_{...}$ are **learnable** weight matrices and $b_{...}$ **learnable** bias vectors.

Encoder-decoder architecture

Sequence to one



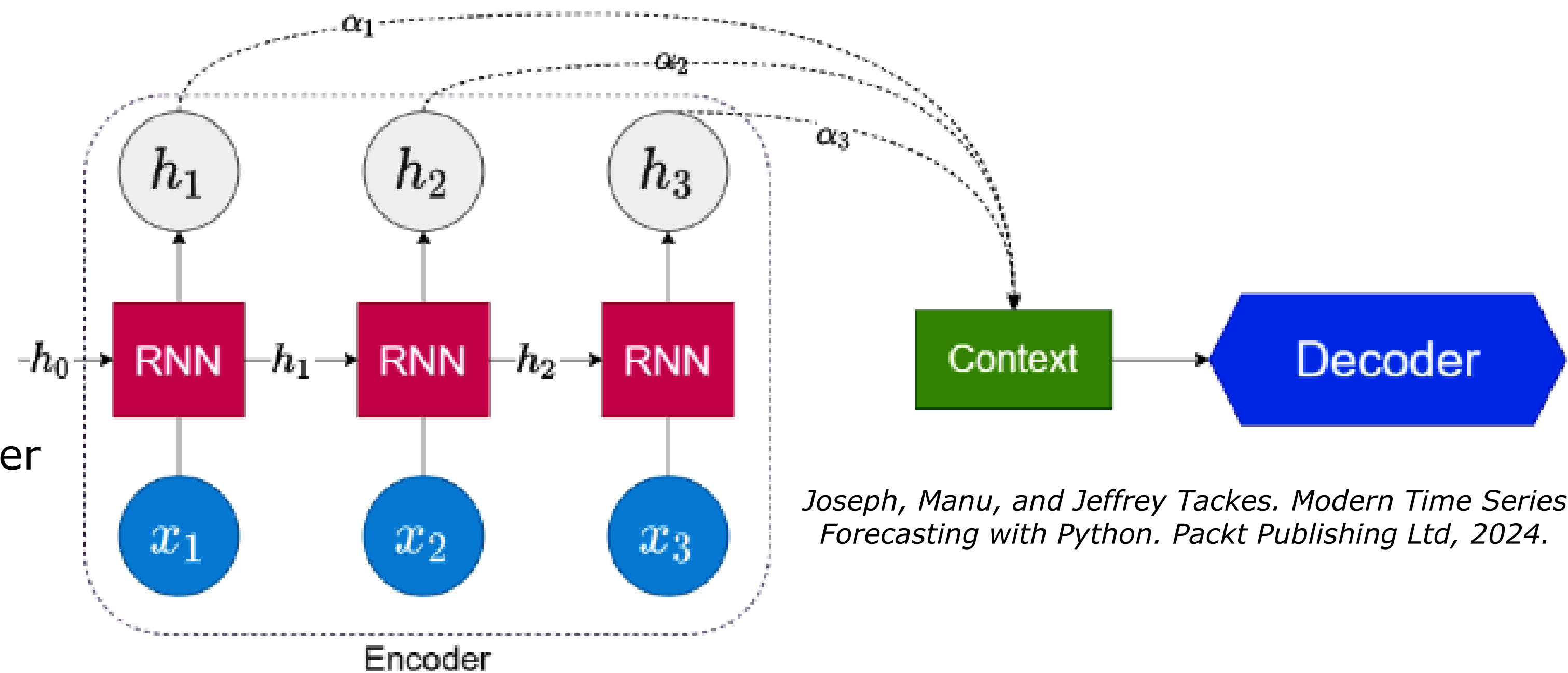
Sequence to sequence



Attention

Using only the last hidden state of the encoder causes an **information bottleneck**.

Instead, learn **attention weights** α_i for each hidden state and combine them into a single **context vector** while decoding.



Generalized attention model: given a set of keys K (e.g., encoder states) and queries Q (e.g., decoder states), an attention model A learns an attention distribution α to be applied to a set of inputs V (typically same as K):

$$A(q, K, V) = \sum_i p(a(k_i, q))v_i$$

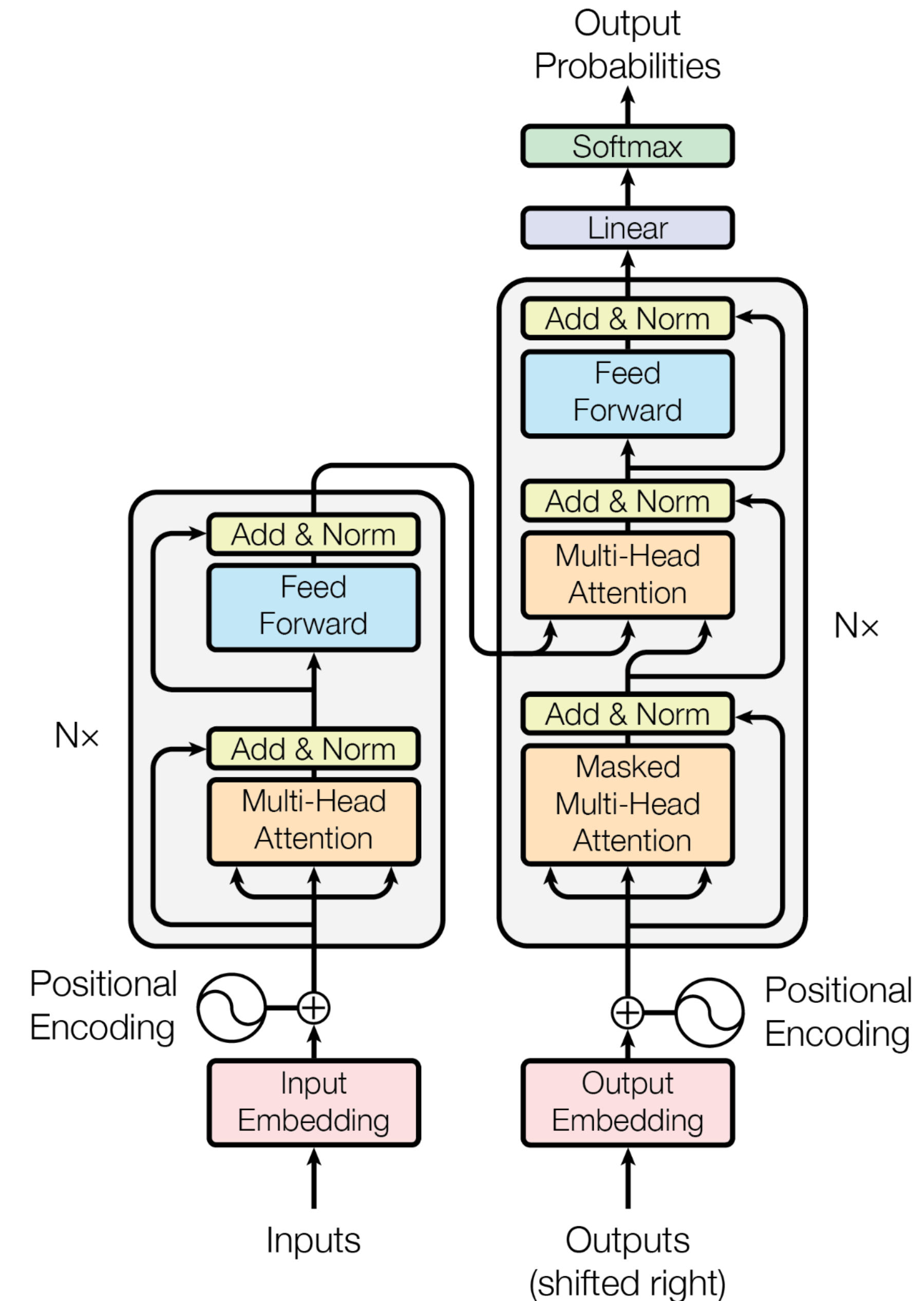
with a an **alignment function** (e.g., dot product) that calculates the **similarity** between queries and keys and p a **distribution function** that converts the similarity score into **attention weights** summing up to 1 (e.g., softmax).

Transformers – Attention is all you need

Due to their recurrence, RNNs cannot be parallelized.

Transformers avoid recurrence altogether and process sequences in parallel.

- **Positional encoding** to retain positional (temporal) information.
- **Skip connections** to improve gradient flow through the network.
- **Layer normalization** for faster training and some regularization.
- **Multi-head self-attention (MHA)**: attention scores between input sequence and itself.
- **Masked MHA**: mask information from the future timesteps.



Improve model performance – Strategies

Reassess ml & application goals

- Are ML objectives **well-aligned** with the actual modeling goals?
- Are the metrics relevant?
- Perform a **qualitative analysis**

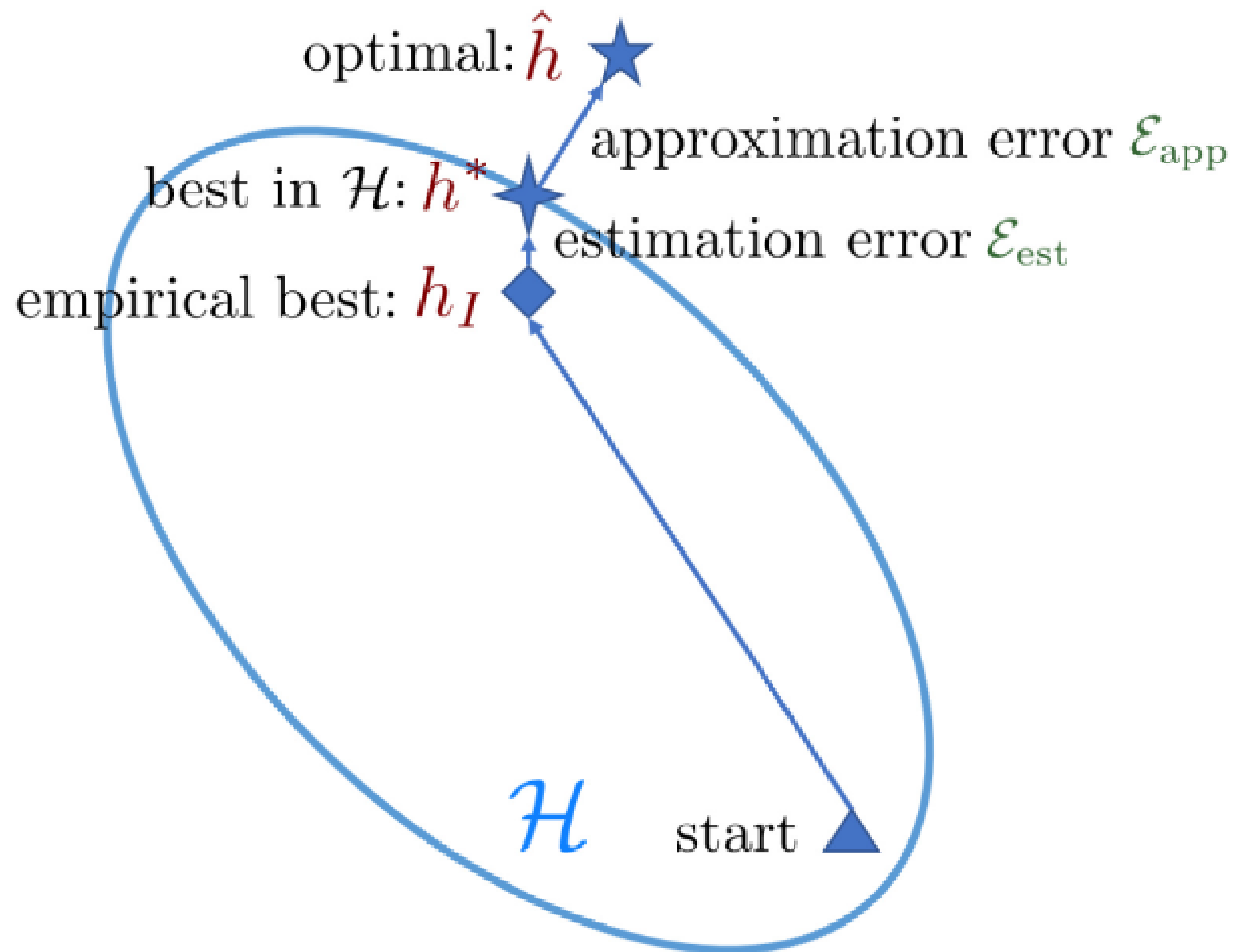
Leverage **expert knowledge**

- Restrict problem complexity
- Interpret model strengths & weaknesses
- Analyze model features

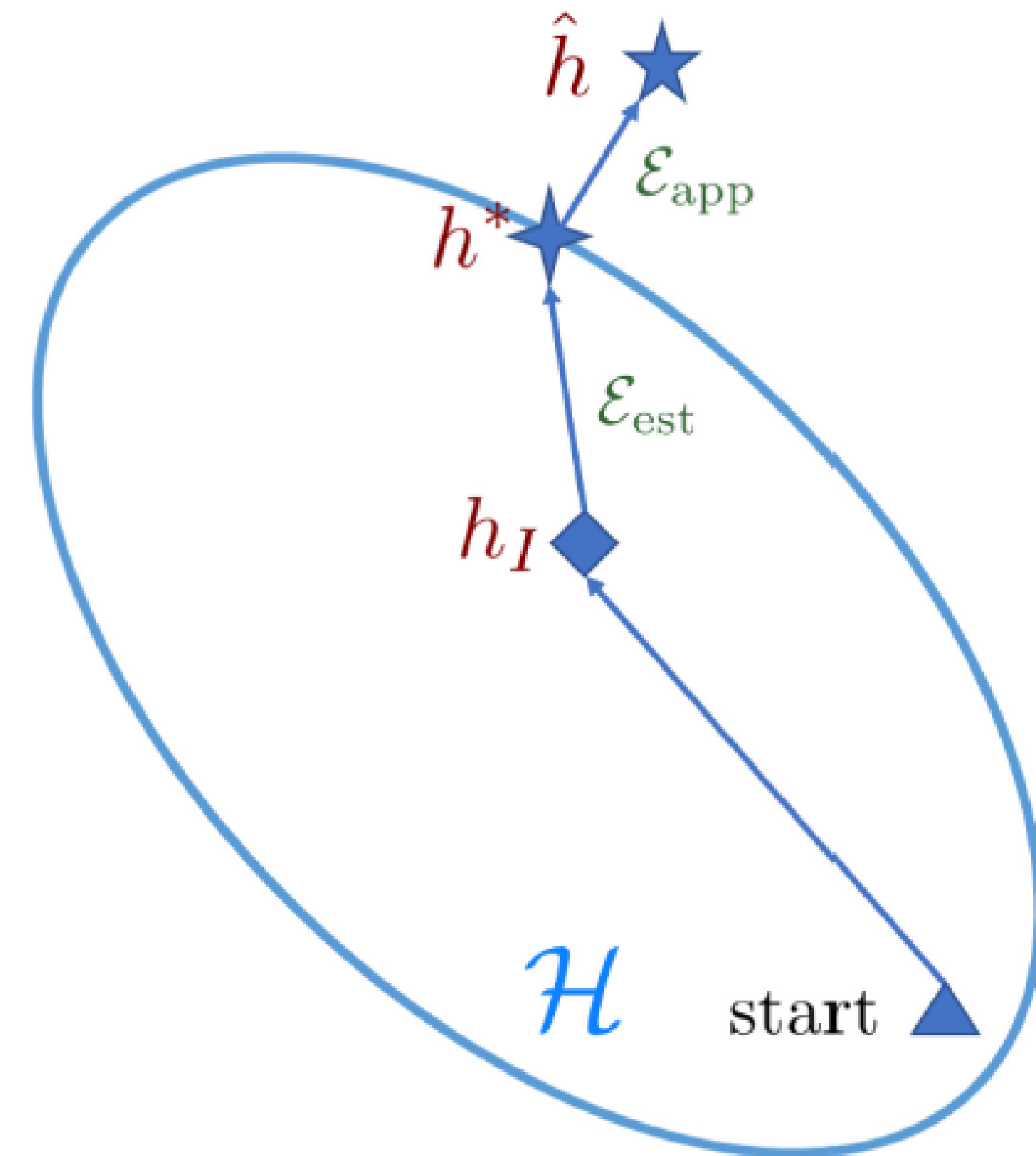
Engineering approach

- Analyze existing approaches and results
- Model tuning (hyperparameters, loss, etc)
- (Model ensembling)

Data scarcity settings



(a) Large I



(b) Small I

Fig. 1. Comparison of learning with sufficient and few training samples.

Mitigating data scarcity using prior knowledge

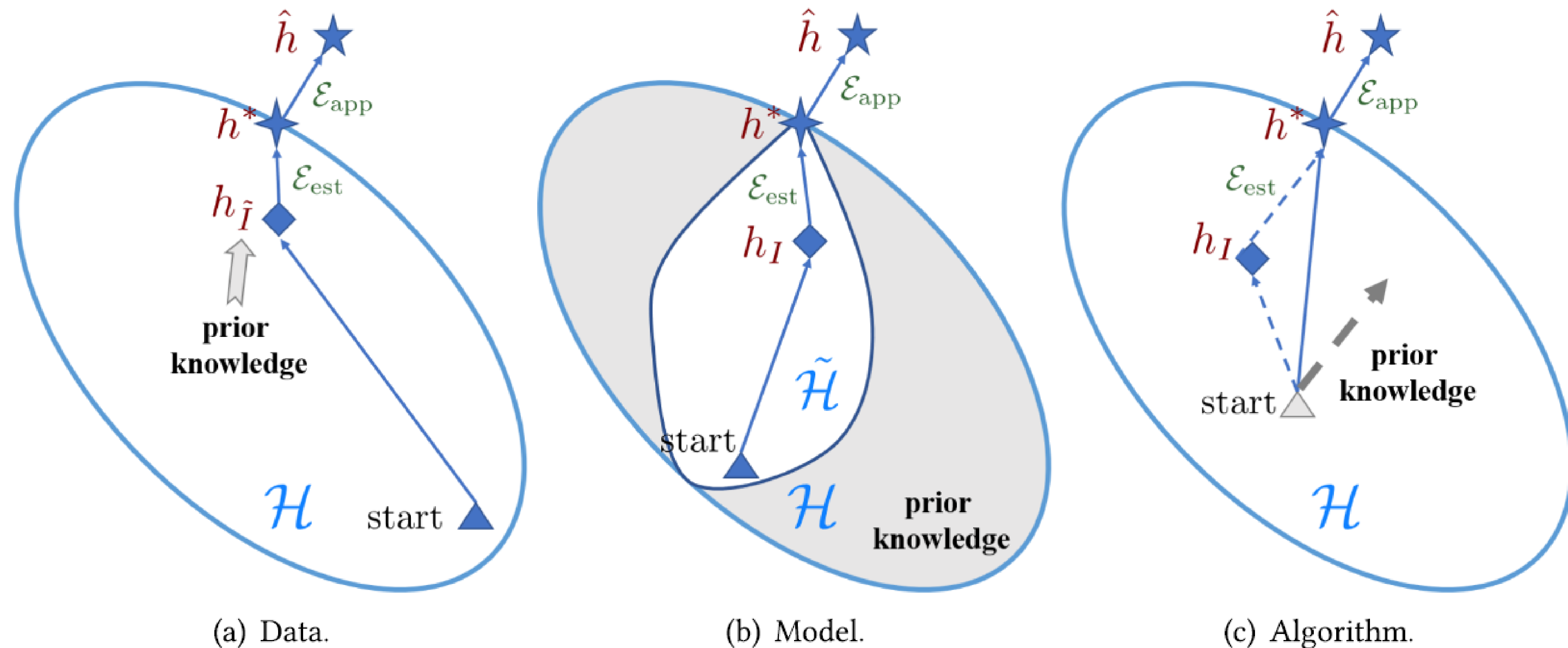
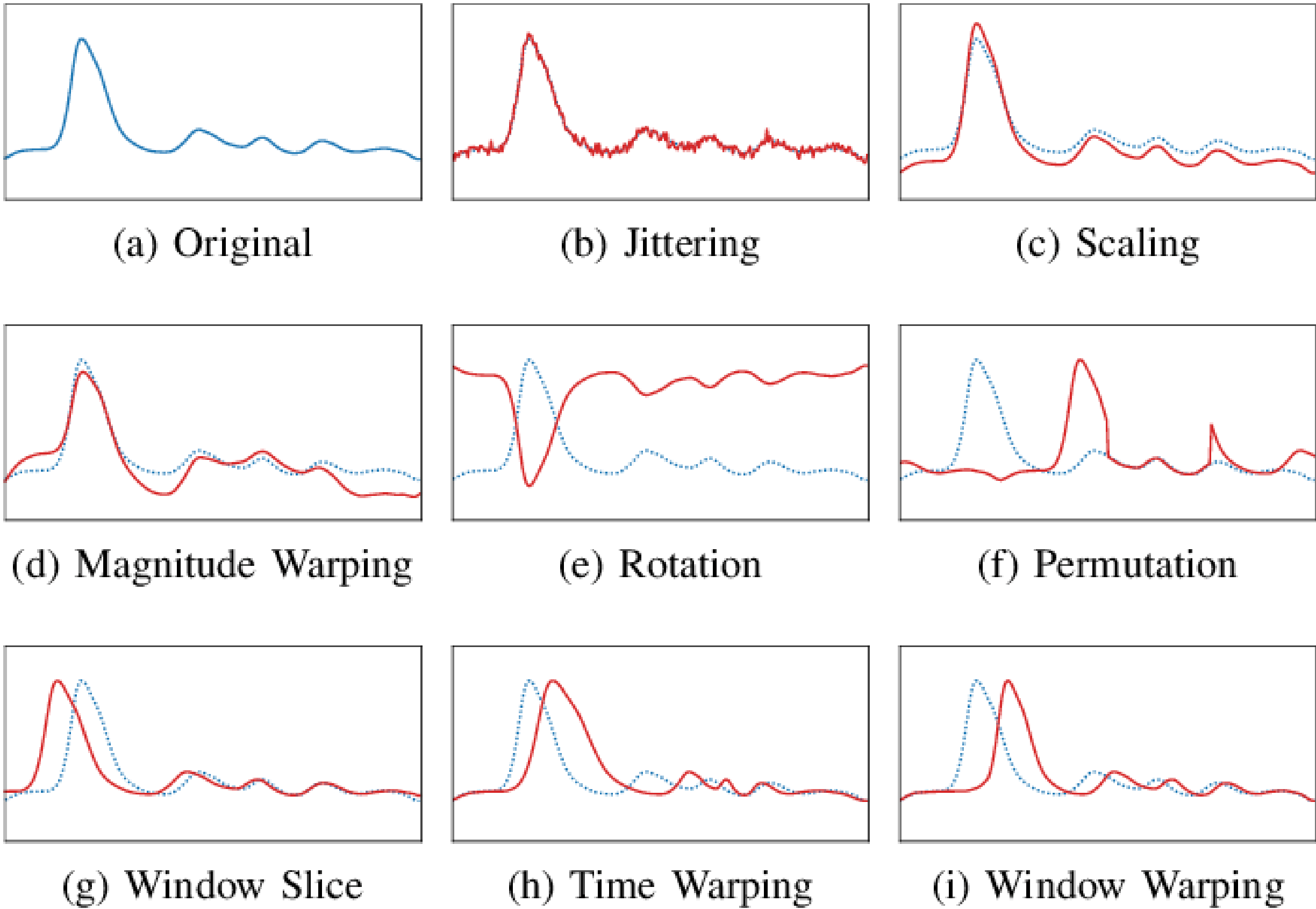
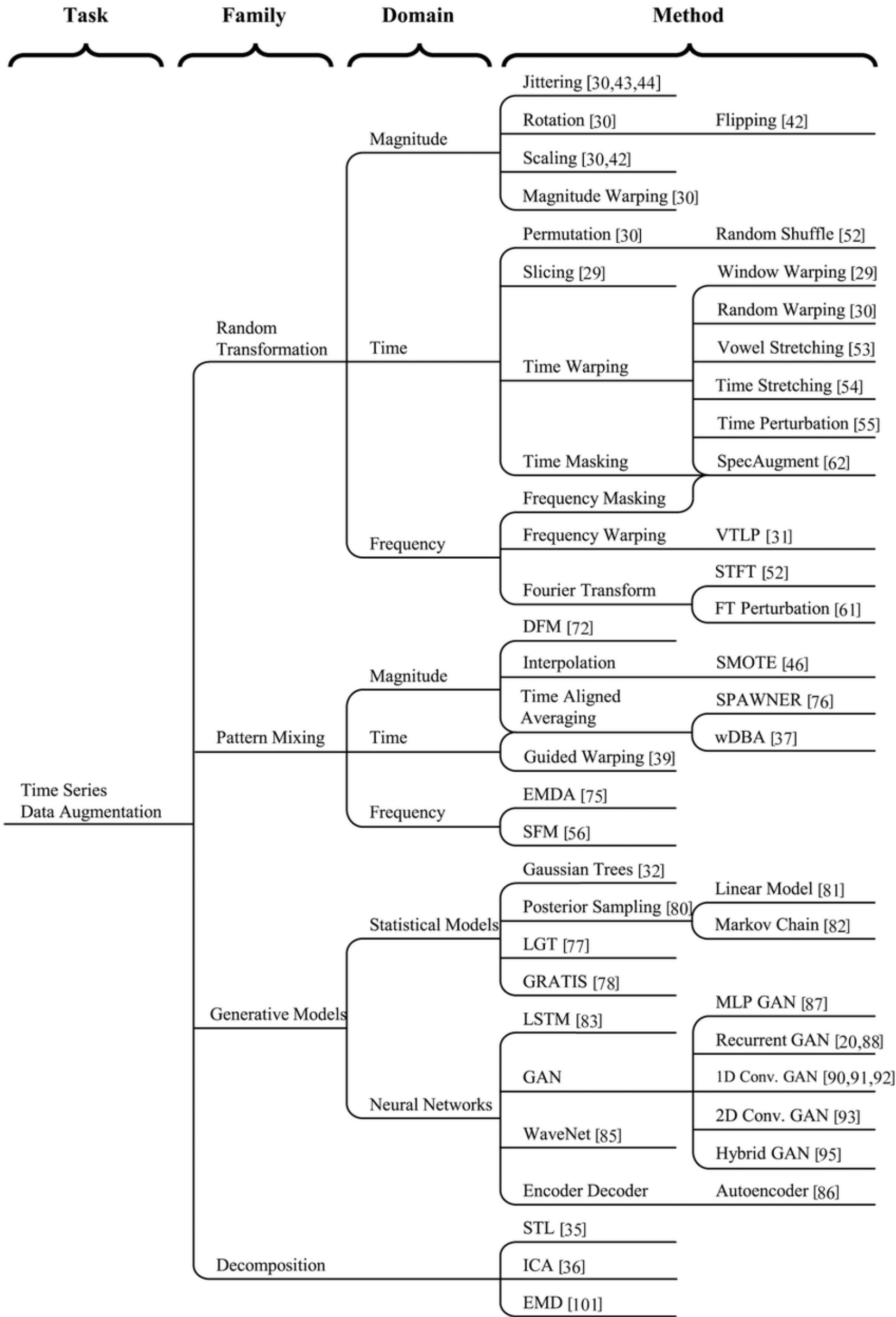


Fig. 2. Different perspectives on how FSL methods solve the few-shot problem.

Data augmentation



Iwana et al. "Time series data augmentation for neural networks by time warping with a discriminative teacher." 2020 25th International Conference on Pattern Recognition (ICPR)



Iwana et al. "An empirical survey of data augmentation for time series classification with neural networks." Plos one 16.7 (2021) Slide 42

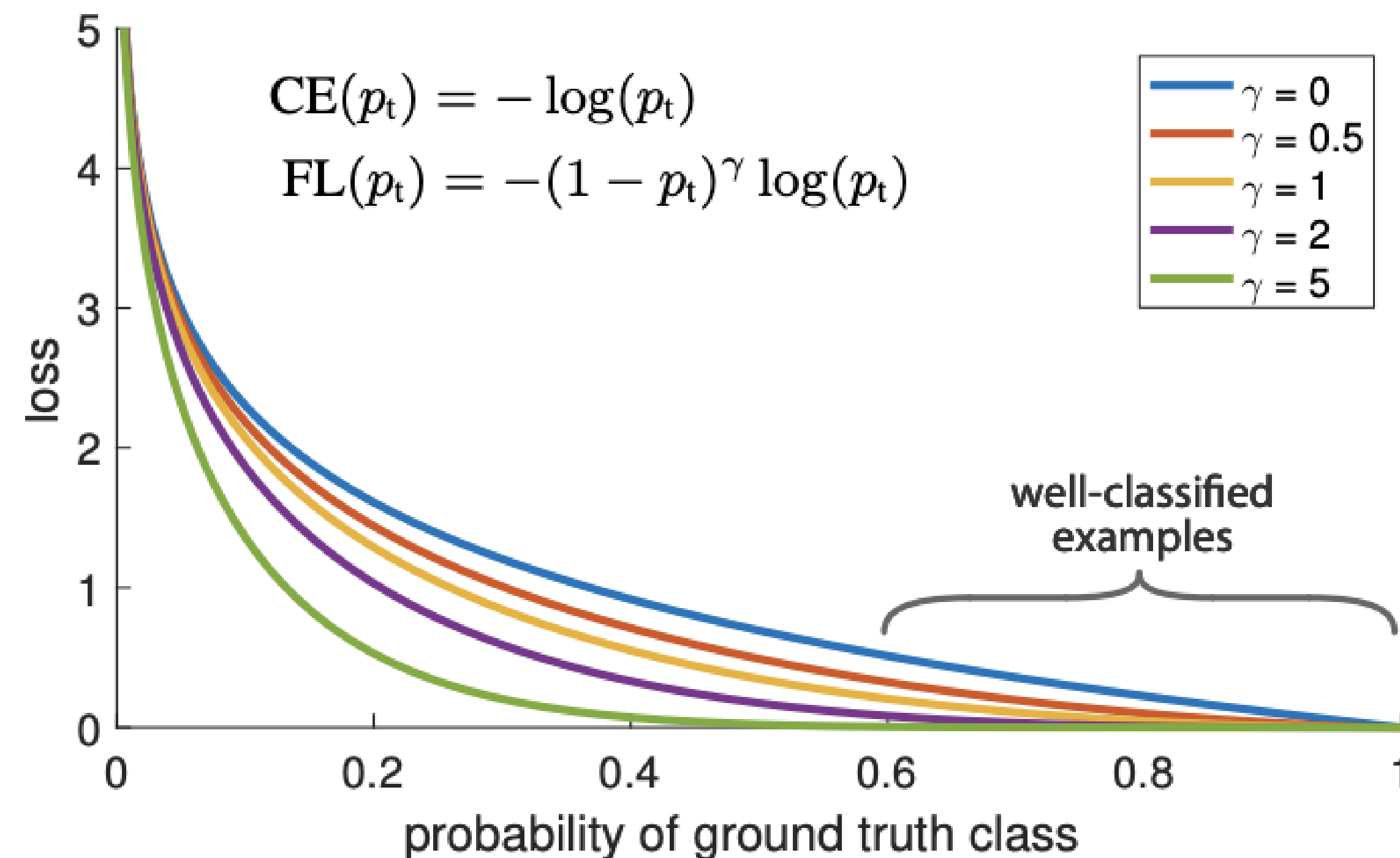
Imbalanced dataset – Loss

Focus on **rare classes**:

- Standard loss $L(X, \theta) = \sum_i L(x_i, \theta)$
- Weighted loss $L(X, \theta) = \sum_i W_{y_i} L(x_i, \theta)$ with $W_c = \frac{|X|}{|\{x_i \in X \mid y_i = c\}|}$

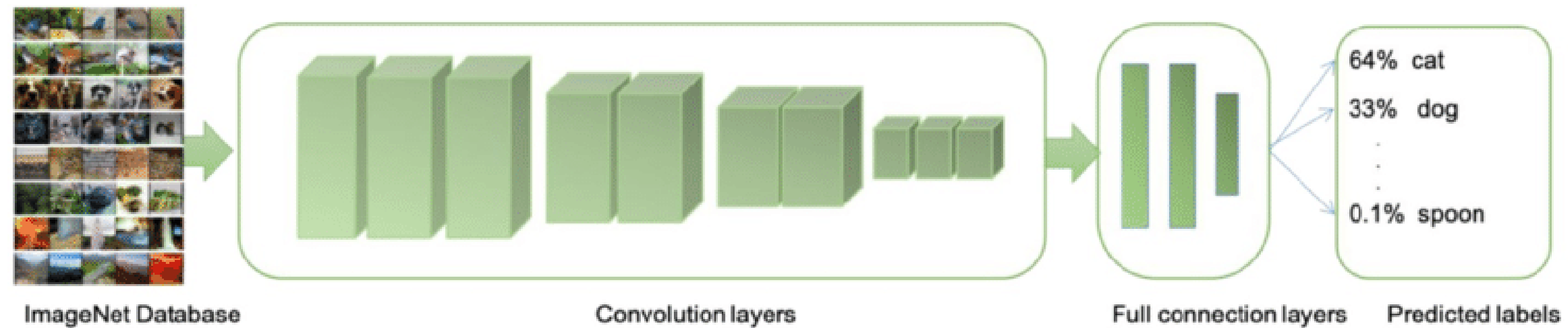
Focus on **hard samples**:

Focal loss, Lin et al. 2018

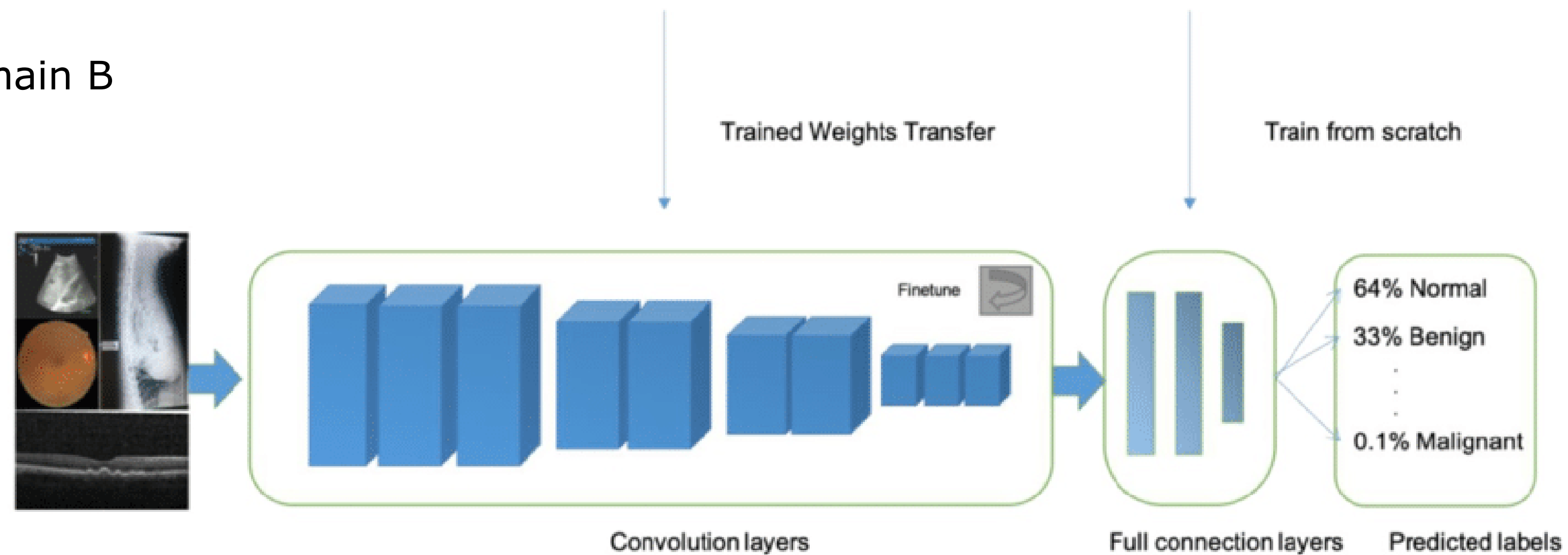


Transfer learning

Transfer knowledge learned on task T1 on domain A



to task T2 on domain B



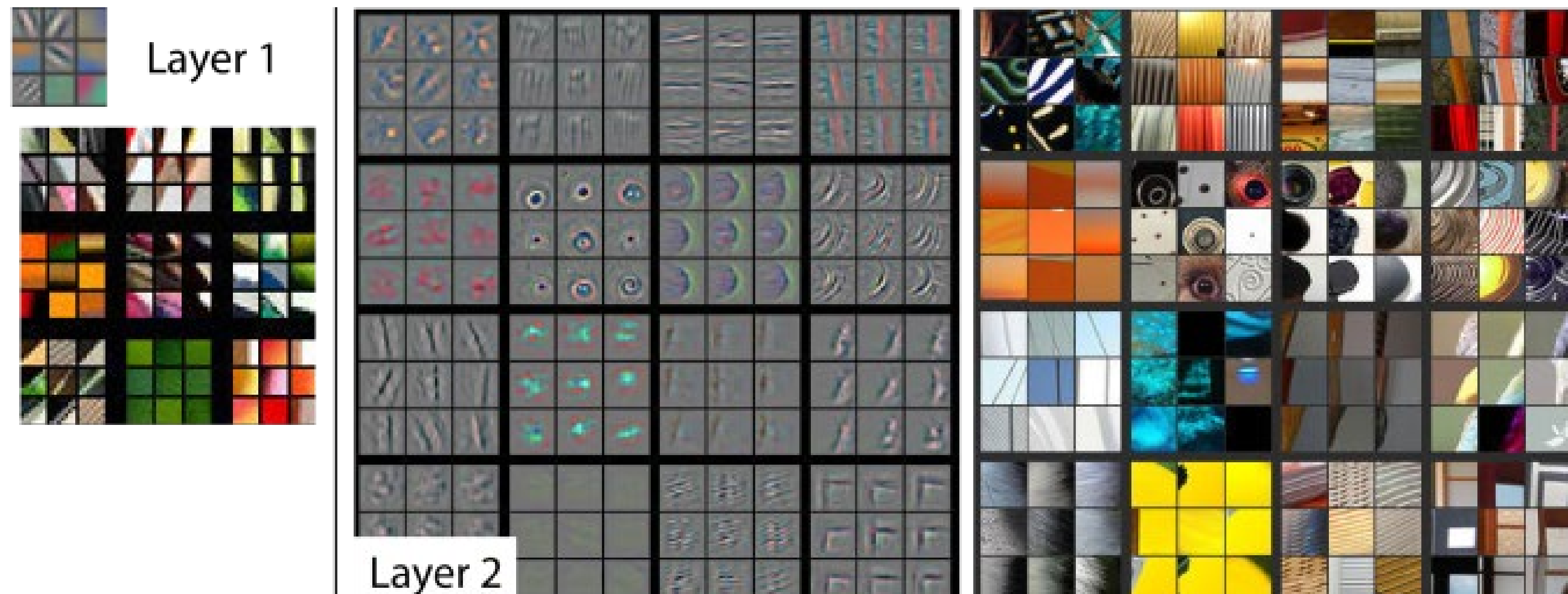
Transfer learning in practice

Load feature extraction backbone with pretrained weights

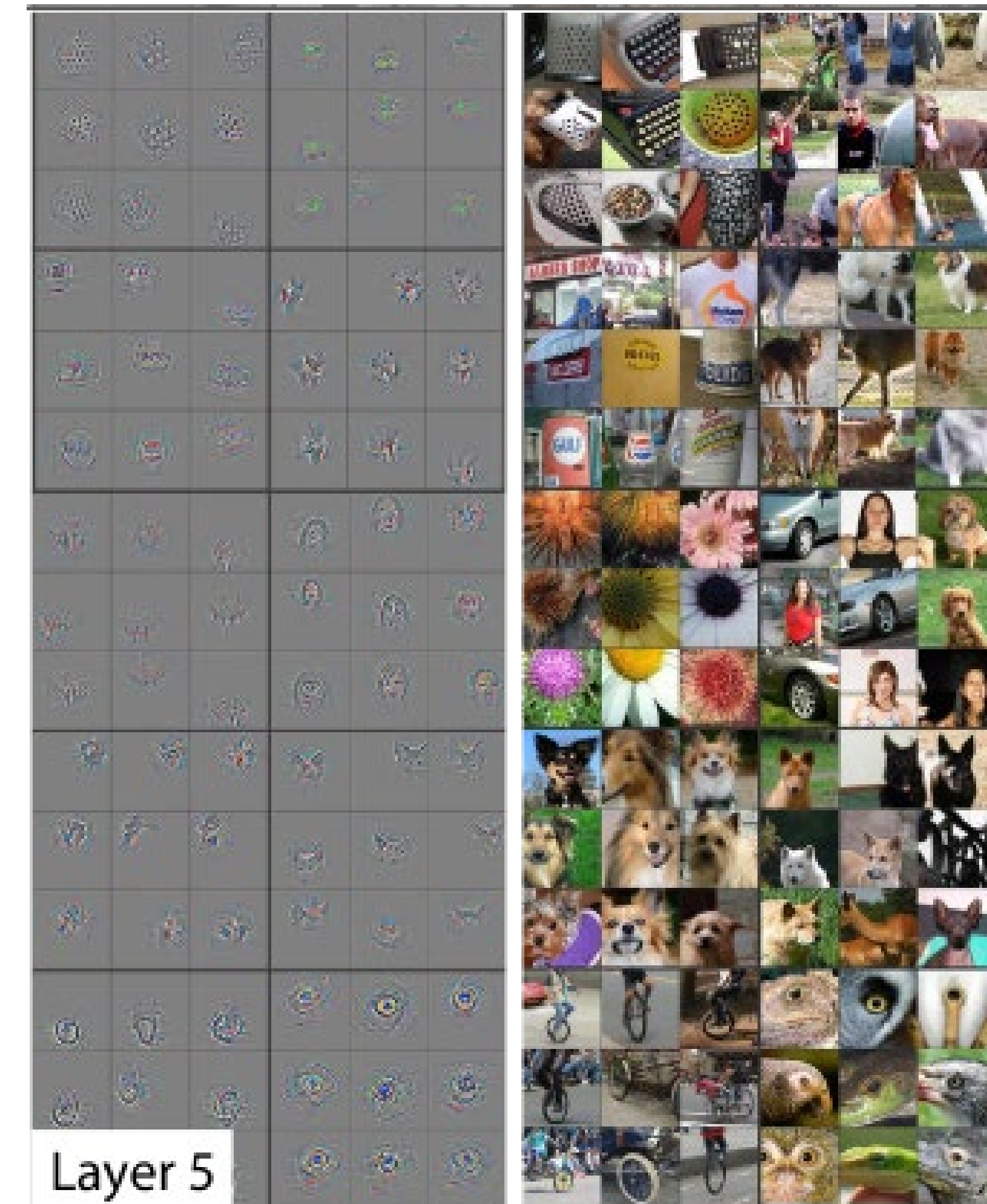
Append classification head to fit your task

- Train new head with frozen pretrained layers

Fine-tuning: unfreeze pretrained layers and retrain model



Zeiler et al. 2013



Multi-task learning

Train model on multiple **related tasks** simultaneously.

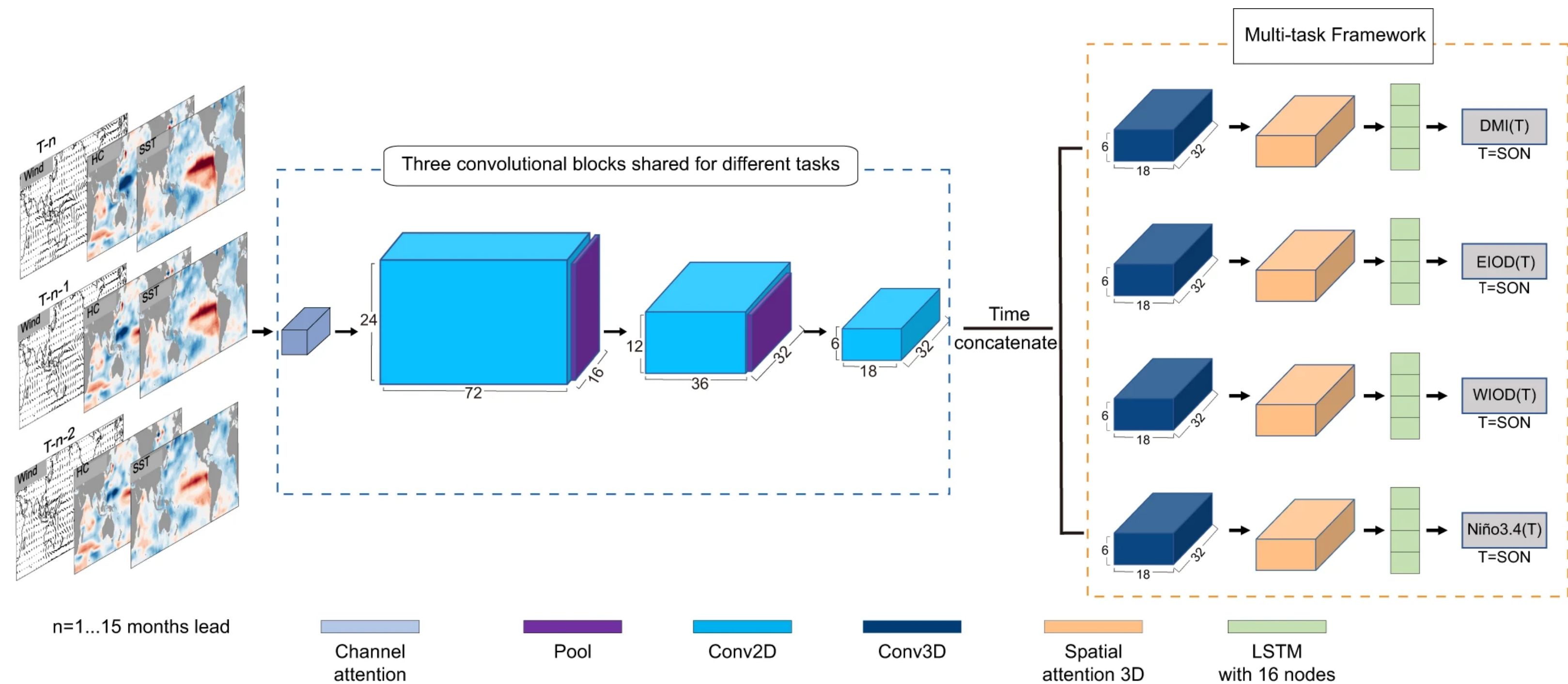
Aggregate task losses

- **Learn** weighting scheme

Share feature representation to improve generalization (regularization effect)

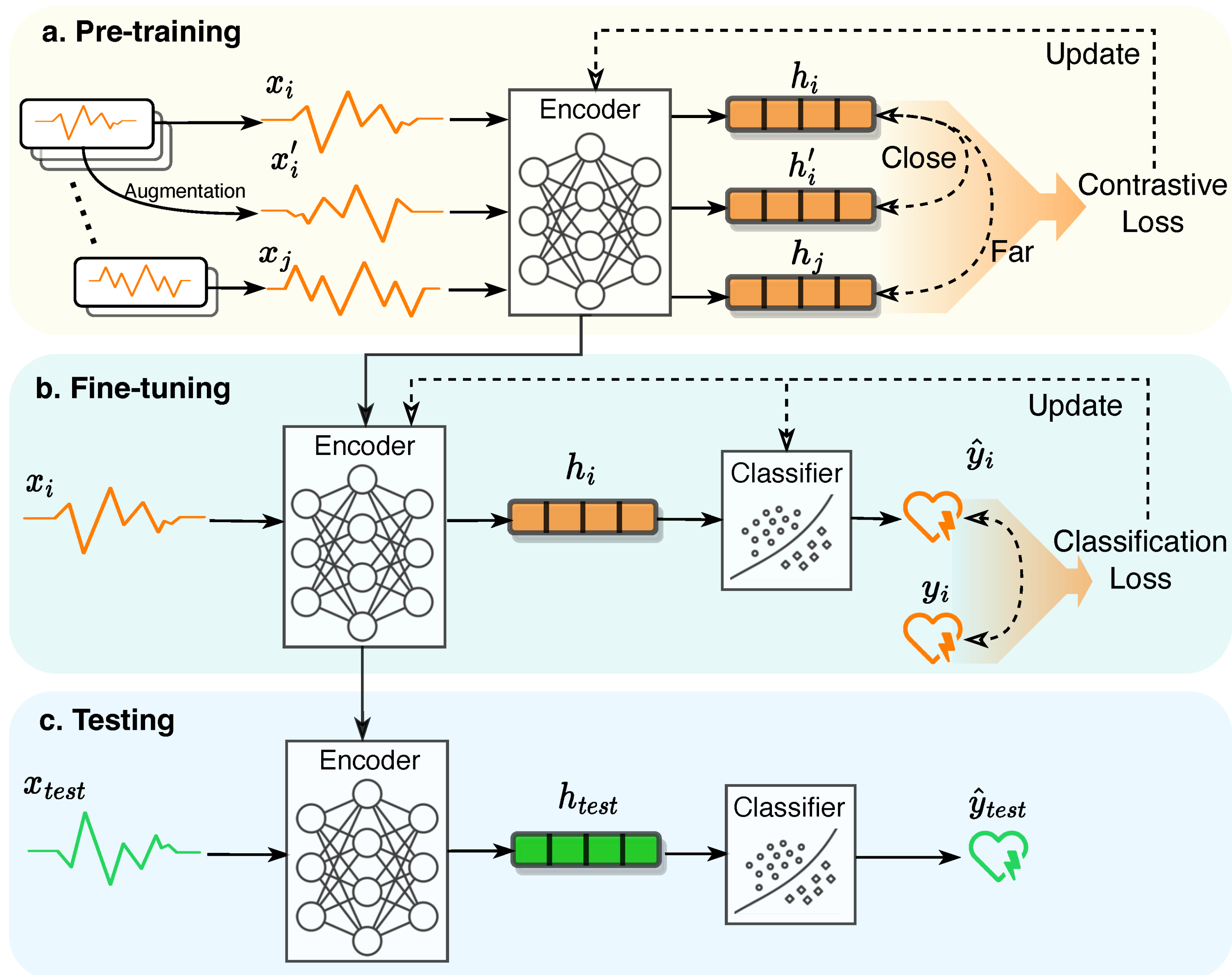
- Hard parameters sharing
- Soft parameters sharing (enforce similarity)

Improve **data efficiency** and reduce computation requirements at inference.



Ling, Fenghua, et al. "Multi-task machine learning improves multi-seasonal prediction of the Indian Ocean Dipole." *Nature Communications* 13.1 (2022): 7681.

Self-supervised learning

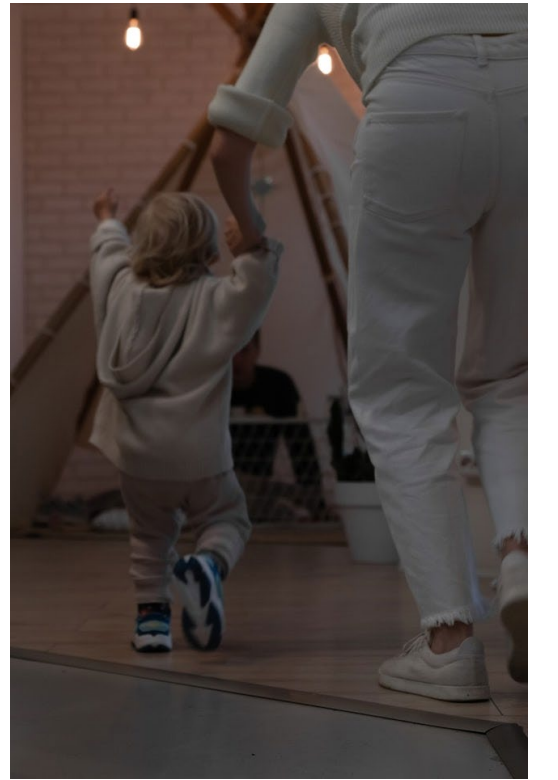


Liu, Ziyu, et al. "Self-supervised contrastive learning for medical time series: A systematic review." *Sensors* 23.9 (2023): 4221.

DL In practice – Starting small

Start with **minimal complexity**

- Small input size
- Small, validated training set (use benchmark datasets such as CIFAR
- Normalize input
- No data augmentation
- Simple baseline model
- Adam optimizer
- Simple training objective
- Minimal codebase



DL In practice – Debugging



Does it **run**?

- Programming bugs ☺
- Wrong tensor shapes
- Wrong tensor types
- Out of GPU memory

Does it **learn**?

- Can you overfit the training data?
- Incorrect loss (e.g. activated logits instead of logits, numerical instabilities, use tf.math operations)
- Is the learning rate too high? Too low?
- Has the model sufficient capacity?
- If you used a benchmark dataset, compare with published results

DL In practice – gradual increase in complexity

1. Set actual training objective
2. Add data augmentation
3. Increase model complexity and regularization
4. Tune hyper parameters: manual/grid/random/Bayesian search
 - Check “A disciplined approach to neural network hyper-parameters”, Leslie 2018
5. Finally, use training callbacks: reduce lr on plateau, early stopping, etc



Josh Tobin, 2019

Hyperparameter	Approximate sensitivity
Learning rate	High
Optimizer choice	Low
Other optimizer params (e.g., Adam beta1)	Low
Batch size	Low
Weight initialization	Medium
Loss function	High
Model depth	Medium
Layer size	High
Layer params (e.g., kernel size)	Medium
Weight of regularization	Medium
Nonlinearity	Low

Doing more with less: optimizing GPU memory usage

Tune input size & batch size

Optimizer choice: Adam maintains two states per model parameter

- SGD, Adafactor, quantized Adam

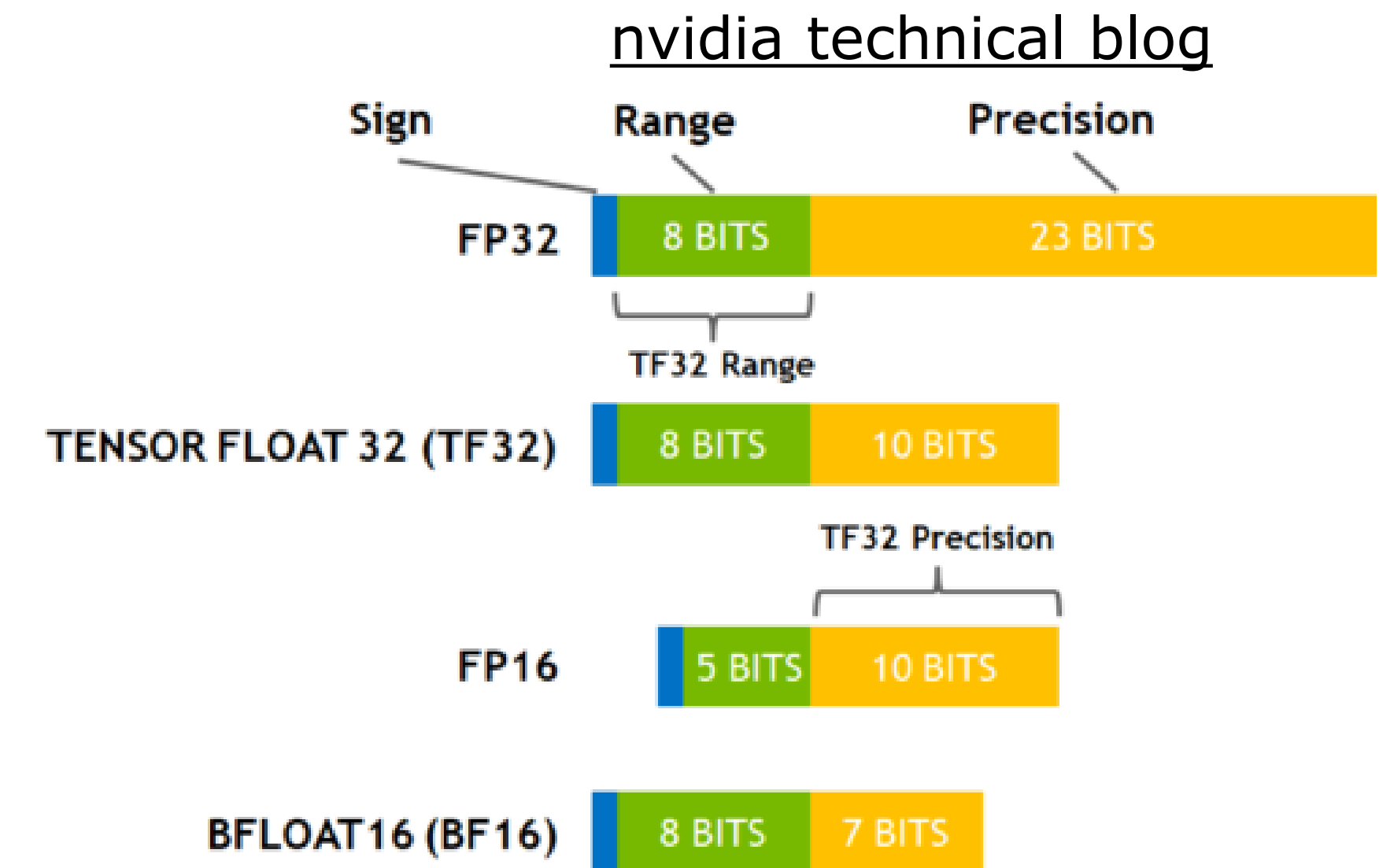
Mixed precision training: computational speedup + enables larger batch size

- Activations are stored with half precision
- Model weights and gradients are stored in both half and full precision

Gradient accumulation: multiple forward/backward passes with smaller batch size before optimization step

Gradient checkpointing: selectively recompute activations during backward pass rather than storing them

- *"Fit 10x larger neural nets into memory at the cost of an additional 20% computation time."*, check [article](#).



Exercise

Review the following resources

- Kaggle tutorial [Intro to Deep Learning](#).
- [Understanding LSTM Networks](#) blog post from Christopher Olah.
- [Troubleshooting Deep Neural Networks](#) by Josh Tobin
- [A Recipe for Training Neural Networks](#) by Andrej Karpathy
- [Practical Advice for Building Deep Neural Networks](#) by Matt & Daniel

Model real-world time series.

- Train time series DL models.
- Make data stationary and retrain models.
- Compare with statistical and ML time series models.